MYDASH · SPEC 055

Sales Cycle Master Spec

Opportunity → proposal → contract → production → billing → commission → renewal

Document	MyDash Sales Cycle Master Spec, v1
Audience	Claude Code (IDE) — implementation agent
Primary user lens	Sales Rep (Dana, Christie); Publisher oversight secondary
Goal	Replace Newspaper Manager end-to-end with feature parity + improvements
Baseline migration	118 (self_serve_creative_workflow)
New migrations	119 through ~130
Integrations assumed live	QuickBooks, Stripe, Gmail, Google Calendar
Primary database	Supabase project hqywacyhpllapdwccmaw
Codebases affected	MyDash (React/Vite) only; StellarPress out of scope
Prepared for	Andrew Jackson, Editor-in-Chief, 13 Stars Media

How to use this document

Read Sections 1 and 2 end-to-end before touching code.

Sections 3–8 cover the six gaps, each self-contained with schema, RPC signatures, UI requirements, and acceptance criteria.

Section 9 lists migrations in apply order. Apply one at a time, test, commit.

Section 10 is the build sequence — follow it unless you spot a dependency issue.

Nothing in here replaces the existing codebase. Build additively and preserve the 41K historical sales.

1. Executive Summary

This spec closes the six gaps between the current MyDash sales cycle and full Newspaper Manager replacement. The audit that follows confirms the codebase is substantially further along than expected — roughly 70% of the cycle is wired. The remaining 30% is structural, not incremental.

Six gaps, in build order:

- **Gap 1 —** Opportunity and typed interactions. Promote opportunity from a sale row in Discovery stage to a first-class entity with its own stage machine, disposition codes, and a Gmail-ingested interaction log. This is the foundation of the CRM and the hardest to retrofit later, so it goes first.
- **Gap 2 —** Press-to-invoice trigger. The billing cascade is specced and the invoices exist in draft state; the trigger that flips them to sent when an issue is marked to press is missing. Small surface, huge leverage.
- **Gap 3 —** Renewal automation. Window widens from 30 to 60 days, status auto-flip, AI-drafted renewal proposal generator, rep surfacing.
- **Gap 4 —** Commission payout execution. 33,555 ledger entries are earned; zero have ever been paid. The UI exists, the logic exists, the publisher needs a confident "pay this" button.
- **Gap 5 —** Sales Rep dashboard. The anchor view for Dana and Christie. Consumes every entity above. Blocked only on Gap 1.
- **Gap 6 —** Integration wiring. QuickBooks invoice/payment sync, Stripe auto-charge, Gmail thread ingestion, Calendar follow-up scheduling. Each has partial schema; none are live end-to-end.

1.1 Current wired state

Migration history runs through 118. The sales cycle has been built incrementally across the last ~30 migrations. Live counts in production confirm what is exercised versus scaffolded.

Clients	4,030 total · 4,028 Active · 0 Renewal · 2 other
Sales	41,090 (historical import; status Closed)
Proposals	0 (new flow never used in production)
Real contracts	0 (historical sales bypass the proposal → contract path)
Invoices	39,725
Payments	28,175
Commission ledger entries	35,825
Commission earned	33,555
Commission paid	0

Ad projects	0 (created on proposal conversion; none yet)

What the counts tell us

The historical import (41,090 sales) seeded the system with Closed sales directly, skipping the proposal → contract flow. That was the correct choice at the time — backfilling proposals retroactively would have no business value.

The commission engine has computed 35,825 ledger entries and moved 33,555 to earned, proving the two-factor trigger (issue published AND invoice paid) works. The payout RPC has never been run — this is an operational gap, not a build gap.

The 0 Renewal clients is the smoking gun for Gap 3: the auto-status logic uses 30 days, but no client with a contract end date inside that window shows up because contract_end_date is populated for synthetic historical contracts that have already lapsed. Widening to 60d alone won't fix it; the Renewal computation also needs to handle the "active-but-winding-down" case properly.

1.2 Full cycle audit

Every node in the sales cycle, labeled by actual state. WIRED means the table/RPC/UI exists and has been exercised. PARTIAL means it exists but has a known gap. GAP means it does not yet exist.

Phase 1 — Opportunity & proposal

Component	Status	Notes
clients table + lifecycle	**WIRED**	4,030 clients, auto-computed status (Lead/Active/Renewal/Lapsed), industry tags, billing block
client_contacts	**WIRED**	Primary + additional contacts with role, email, phone
Opportunity entity	**GAP**	Currently masquerades as a sale row in Discovery/Presentation/Proposal/Negotiation stages
Typed interactions (calls/emails/meetings)	**GAP**	activity_log exists but is untyped; no Gmail/Calendar ingestion
proposals + proposal_lines	**WIRED**	Full schema with art_source, charge_day, delivery cadence, brief fields
proposal_signatures	**WIRED**	Access tokens, view tracking, snapshot capture, expires_at
Proposal PDF generation	**PARTIAL**	HTML email template ready; PDF generator referenced but not confirmed live

Phase 2 — Contract conversion

Component	Status	Notes
convert_proposal_to_contract RPC	**WIRED**	Atomic: creates contracts, contract_lines, sales, ad_projects, threads, invoices

Component	Status	Notes
contracts + contract_lines	WIRED	Flight dates, page/RH/placement guarantees, monthly_amount, charge_day
cancel_contract RPC	WIRED	Cancellation path exists
Contract detail UI	PARTIAL	Displays but not the renewal launch point it needs to be

Phase 3 — Production

Component	Status	Notes
ad_projects	WIRED	Full lifecycle, art_source, revision tracking, signoff, global/project assets
tr_ad_project_signoff_creates_placement	WIRED	Trigger creates placement on dual signoff
Revision billing (revisions 4+)	PARTIAL	Counter exists; auto-invoice on approval needs trigger
Camera-ready fast-track	WIRED	Status progression: awaiting art → art received → approved → placed
Self-serve ad bookings	WIRED	Parallel system: ad_bookings, creative_jobs, ad_placements — out of scope for this spec

Phase 4 — Billing

Component	Status	Notes
invoices (per-issue)	WIRED	Drafted by convert_proposal_to_contract; reminders, Stripe + QBO sync fields
Press-to-invoice trigger	GAP	issues.sent_to_press_at does not currently flip invoices draft → sent
generate_pending_invoices RPC	WIRED	Magazine lead-day-based bulk creation
payments + apply_payment_to_invoices	WIRED	Oldest-unpaid-first waterfall, credit_balance overflow
invoices_auto_mark_paid trigger	WIRED	Marks invoice paid when balance reaches 0
Reminder cadence sends	PARTIAL	Columns exist (first_reminder_sent, etc.); scheduler job missing
Stripe auto-charge on invoice send	PARTIAL	stripe_payment_intent_id, auto_charge_attempts columns exist; Edge Function missing
QuickBooks invoice sync	PARTIAL	quickbooks_id columns exist; sync worker missing
Credit hold	WIRED	credit_hold + reason + audit on clients

Phase 5 — Commission

Component	Status	Notes
commission_rates (rep × pub × product)	**WIRED**	Full matrix with 20% default
commission_issue_goals	**WIRED**	Per-issue goal with share % per rep
salesperson_pub_assignments + shares	**WIRED**	Two-factor trigger config per rep
commission_ledger (two-factor)	**WIRED**	35,825 entries, 33,555 earned
calculate_sale_commission RPC	**WIRED**	Computes entry; bonus tier applied at period close
recalculate_all_commissions RPC	**WIRED**	Bulk refresh
commission_payouts + payout run	**PARTIAL**	Schema exists; UI workflow for publisher confirmation incomplete — 0 paid
QuickBooks payroll sync	**GAP**	Not yet wired; future phase

Phase 6 — Renewal

Component	Status	Notes
contract_end_date on clients	**WIRED**	Populated from latest contract line
proposals.renewal_date	**WIRED**	Column exists
Renewal status auto-flip (60d window)	**GAP**	Current logic uses 30d; nothing computing off renewal_date
AI-drafted renewal proposal	**GAP**	ai_drafted_proposal_text exists on sales; no generator
Renewal queue UI	**GAP**	Blocked on status logic + rep dashboard

1.3 Scope boundaries

What this spec does NOT cover, to keep the build focused:

- StellarPress — public-facing reader is out of scope entirely. MyDash-only.
- Historical data backfill — the 41,090 legacy sales stay as-is. Only new sales flow through the new path.
- Self-serve ad bookings (ad_bookings, creative_jobs) — a parallel path for direct-client purchases; not the rep-driven cycle.
- Subscription management — separate module, already planned for a later phase.
- Distribution / circulation — separate module.
- Wednesday Agent Station — deferred until core cycle is live.

- Legal notices — separate workflow, already largely wired.

1.4 Acceptance criteria for the full build

Before this spec is considered executed, all of the following must be demonstrably true in a live production environment:

- **Rep creates an opportunity,** logs three interactions (a call, an email auto-ingested from Gmail, a meeting with Calendar sync), sends a proposal, and the client signs it — all without leaving MyDash.
- **Publisher marks an issue sent to press,** and within 60 seconds every invoice on that issue flips from draft to sent, QuickBooks receives the invoice, and clients with cards on file are auto-charged via Stripe.
- **A client with a contract ending in 60 days** shows up on the Sales Rep dashboard under a Renewal tab, with an AI-drafted renewal proposal one click away.
- **Publisher runs a monthly payout,** confirms the total on screen, and the commission_payouts row and ledger entries flip to paid in a single transaction.
- **Every screen Dana sees** is sourced from the opportunities, interactions, proposals, contracts, sales, invoices, commissions tables — no N+1 queries, no manual joins in the UI.

2. Full Cycle Flow Map

The cycle is six phases, four primary actors, and one atomic boundary. Every row entering the system goes through the same path; no sale is created outside this flow (excluding the parallel self-serve booking path and the historical data already imported).

2.1 Actor map

Sales Rep (Dana, Christie)	Owns Phases 1, 6. Creates opportunities, logs interactions, sends proposals, tracks renewals.
Client (advertiser)	Signs the proposal. No other touchpoint in the system itself (emails, clicks on signature link, replies).
Ad Designer (Jen)	Owns Phase 3. Designs, sends proofs, tracks revisions. Already specced in prior Ad Lifecycle doc.
Publisher (Hayley)	Owns press-to-press trigger in Phase 4 and payout run in Phase 5. Oversight on every phase.
Office Admin (Cami)	Phase 4 billing operations: records non-Stripe payments, resolves credit holds, manages collections.
System (RPCs + triggers)	The atomic convert_proposal_to_contract RPC; press trigger cascade; reminder scheduler; commission calculator.

2.2 Phase map — six phases, linearized

Each phase has an entry condition, a primary actor, an action, and an exit condition that feeds the next phase. Phase boundaries are enforced by either a trigger (press, signature) or an auto-computed status (renewal window).

Phase 1 · Opportunity → Proposal

Entry	Client exists (Lead, Active, Renewal, or Lapsed)
Actor	Sales Rep
Action	Create opportunity → log interactions → build proposal → send for signature
Exit	Proposal signed (proposal_signatures.signed = true)
Entities touched	clients, opportunities (new), interactions (new), proposals, proposal_lines, proposal_signatures
Duration	Days to months; Dana may log 15+ interactions per opportunity

Phase 2 · Signature → Contract

Entry	proposal_signatures.signed = true
Actor	System (atomic RPC)

Action	convert_proposal_to_contract fires; creates contract + contract_lines + sales + ad_projects + threads + invoices
Exit	contract row exists with status = active; invoices in draft state
Entities touched	contracts, contract_lines, sales, ad_projects, message_threads, invoices, commission_ledger
Duration	Milliseconds (single transaction)

Phase 3 · Production

Entry	ad_project.status = "brief ready"
Actor	Ad Designer (we-design track) or Client (camera-ready track)
Action	Design → proof → revise → approve. Dual signoff (designer + salesperson) fires placement trigger.
Exit	ad_project.status = approved; print_placement row exists
Entities touched	ad_projects, print_placements, message_threads, revision_billable_count on ad_project
Duration	Days per ad; independent of contract flight dates

Phase 4 · Billing

Entry	issue.sent_to_press_at stamped
Actor	Publisher (press); System (cascade); Office Admin (exceptions)
Action	Flip invoices draft → sent → QuickBooks sync → Stripe auto-charge → reminders → payment → waterfall apply
Exit	invoice.status = paid OR invoice.status = overdue + credit_hold fired
Entities touched	issues, invoices, invoice_lines, payments, clients.credit_balance, clients.credit_hold
Duration	Press day through payment; typically 30-60 days

Phase 5 · Commission

Entry	commission_ledger entry moves to earned (issue_published AND invoice_paid)
Actor	System (calculate); Publisher (payout)
Action	Two-factor trigger → bonus tier applied at period close → publisher runs payout batch
Exit	commission_ledger.status = paid; commission_payouts row = paid
Entities touched	commission_ledger, commission_payouts, commission_issue_goals
Duration	Monthly or bi-weekly payout cadence per rep

Phase 6 · Renewal

Entry	contract.end_date minus 60 days = today
Actor	System (status flip); Sales Rep (action)
Action	Client status flips to Renewal → rep dashboard surfaces → AI drafts renewal proposal → rep reviews + sends
Exit	New proposal sent; loops to Phase 1
Entities touched	clients.status, contracts, proposals (new renewal), opportunities (new)
Duration	60 days to renewal; longer if lapsed and re-engaged

2.3 Data flow — which entity creates what

The row-creation chain is deterministic and only ever flows downstream. Nothing higher in the chain is created by anything lower.

```
clients
  └─ client_contacts
  └─ opportunities (NEW — Gap 1)
       └─ interactions (NEW — Gap 1) — 1:N typed comms
       └─ proposals
            └─ proposal_lines
            └─ proposal_signatures
            └─ [SIGNATURE EVENT — atomic]
                 └─ contracts
                      └─ contract_lines
                      └─ sales (one per issue-ad)
                           └─ ad_projects (one per unique ad size)
                                └─ message_threads
                                └─ print_placements (on dual signoff)
                           └─ commission_ledger (pending)
                      └─ invoices (drafted per issue, charge_day, or lump)
                           └─ invoice_lines
                           └─ [PRESS EVENT — Gap 2]
                                └─ invoice.status: draft → sent
                                └─ Stripe auto-charge (Gap 6)
                                └─ QuickBooks sync (Gap 6)
                                     └─ payments (Stripe or manual)
                                          └─ commission_ledger → earned
                                          └─ [PAYOUT RUN — Gap 4]
                                               └─ commission_payouts
                                               └─ ledger → paid
```

2.4 Transactional boundaries

Three atomic boundaries in the cycle, all currently enforced at the database level:

- **Proposal signature → full contract stack.** One RPC: convert_proposal_to_contract. Already atomic. No change needed.
- **Payment receipt → invoice waterfall.** One RPC: apply_payment_to_invoices. Already atomic. Credit overflow via trigger.
- **Press release → invoice cascade.** Gap 2. Must be atomic: the trigger on issues.sent_to_press_at must flip every invoice associated with that issue, queue QBO sync, and queue Stripe charge in one transaction per invoice (with the external calls happening async via a job queue or Edge Function).

2.5 What rep sees at each phase

This is the UX ordering principle for the Sales Rep Dashboard (Gap 5). The rep moves left-to-right across phases; each column surfaces the actionable items that belong to that rep.

Phase 1	Opportunities tab: stage kanban, interaction log, "send proposal" action
Phase 1 → 2	Proposals tab: sent / under review / signed counts; resend + view signature page actions
Phase 2	Contracts tab: active contract list; contract detail = source-of-truth view for all their accounts
Phase 3	Production tab (read-only): ad projects in progress; proof-sent alerts, revision alerts
Phase 4	Billing tab: collections worklist for their clients only; overdue, credit hold, auto-charge failures
Phase 5	Commission tab: YTD earned, pending earning, bonus tier tracking, next payout date
Phase 6	Renewals tab: 60-day window, 30-day urgent, lapsed-but-reachable; AI-draft button on each

One mental model for everything

Every object in the system belongs to exactly one client. Every action a rep takes is about one client. The dashboard never asks the rep to context-switch — it aggregates across clients but each card is traceable back to a single client profile.

This is why Gap 5 (Sales Rep Dashboard) is downstream of Gap 1 (Opportunity model). Without a first-class opportunity, the dashboard can't show the rep what they're actually working on — only what's already closed.

3. Gap 1 — Opportunity & Interaction Model

The first-class opportunity entity is the foundation of the CRM. It lives between the client and the proposal, owns the pre-signature sales pursuit, and anchors every interaction logged with that client during that pursuit. This gap must be closed before Gaps 5 (dashboard) and 3 (renewal) can be completed cleanly.

Design principle

Two entities, clean separation: opportunities own the pursuit phase; sales rows only exist after signature (in fulfillment). Historical Closed sales (41,090 rows) are untouched; they keep their current shape and are treated as legacy fulfillment records with no parent opportunity.

Gmail auto-ingest matches threads by contact email, files under the active opportunity automatically. No manual linking step in the happy path.

Four enum disposition axes: stage (pursuit position), next_action (what's next), outcome (why it closed), channel (where it came from). Each axis answers a different report.

3.1 Entity specification

3.1.1 opportunities table

The pursuit record. Created when a rep decides to actively work a client. Lives from Discovery through Won (proposal signed) or Lost (abandoned). One opportunity can result in one proposal, one contract, and many sales rows.

Column	Type	Purpose
`id`	`uuid PK`	Primary key, default gen_random_uuid()
`client_id`	`uuid FK → clients`	NOT NULL. Parent client. ON DELETE RESTRICT.
`name`	`text`	Rep-defined label, e.g. "Kennedy Club Fall Campaign". NOT NULL.
`stage`	`opportunity_stage enum`	Discovery, Qualifying, Proposal, Negotiation, Won, Lost. Default Discovery.
`source_channel`	`opportunity_channel enum`	inbound_call, outbound_call, email, meeting, referral, walk_in, web_form, event, renewal, other. NOT NULL.
`outcome`	`opportunity_outcome enum`	Nullable; required when stage = Lost. qualified_not_ready, budget, competitor, no_response, timing, product_fit, other.
`assigned_to`	`uuid FK → team_members`	Primary rep owning pursuit. NOT NULL.
`estimated_value`	`numeric(10,2)`	Rep estimate at creation; drives pipeline forecast weighting.
`weighted_value`	`numeric(10,2)`	Auto-computed: estimated_value × stage_weight (see 3.1.4).

Column	Type	Purpose
expected_close_date	date	Rep estimate; drives Calendar follow-up and aging reports.
actual_close_date	date	Stamped on transition to Won or Lost.
publications_of_interest	text[]	Array of publication IDs; pre-populated from client.interested_pubs.
products_of_interest	text[]	Array of product_type values for filtering and AI draft priming.
competitor_client_id	uuid FK → clients	Nullable; if rep knows the competitor, link for industry context.
proposal_id	uuid FK → proposals	Set when rep builds a proposal from this opportunity. Nullable.
contract_id	uuid FK → contracts	Set when proposal converts. Nullable.
lost_reason_detail	text	Free-text when outcome is selected; optional.
priority	smallint	Rep-defined 1-5 (5 = highest). Default 3. Drives dashboard sort.
tags	text[]	Free-form tags for rep organization (e.g. "special-section", "wine-festival").
notes	text	Rolling notes field; separate from interactions.
last_interaction_at	timestamptz	Auto-updated by trigger when an interaction is logged.
created_at	timestamptz	Default now().
updated_at	timestamptz	Auto via tr_opportunities_updated.
created_by	uuid FK → team_members	Not necessarily the assignee (publisher can create + assign).

3.1.2 interactions table

Typed communications log. One row per call, email thread, meeting, or text. Gmail auto-ingest creates rows automatically; reps can also log manually. Replaces the untyped activity_log for sales contexts (activity_log stays for system audit).

Column	Type	Purpose
id	uuid PK	Primary key.
opportunity_id	uuid FK → opportunities	Nullable — some interactions happen before an opportunity exists (cold call). If null, client_id must be set.
client_id	uuid FK → clients	NOT NULL. Redundant with opportunity.client_id when opportunity is set, but simplifies queries.
type	interaction_type enum	call, email, meeting, text, note, task, voicemail, dm. NOT NULL.
direction	interaction_direction enum	inbound, outbound, internal. NOT NULL.
occurred_at	timestamptz	When the interaction happened (not when it was logged). NOT NULL.

Column	Type	Purpose
duration_minutes	integer	For calls/meetings. Nullable.
subject	text	Email subject, call topic, meeting title. Nullable for texts.
body	text	Free-text summary or quote; Gmail ingestion stores stripped email body.
contact_id	uuid FK → client_contacts	Specific contact person on the other end. Nullable.
contact_email	text	Snapshot of email address — helps with Gmail matching.
participant_ids	uuid[]	Other team members present (meetings, CCs).
disposition	interaction_disposition enum	completed, no_answer, left_voicemail, rescheduled, cancelled, no_show. Nullable.
source	interaction_source enum	manual, gmail_ingest, calendar_ingest, phone_log, text_ingest. NOT NULL, default manual.
external_id	text	Gmail message ID, Calendar event ID, etc. Unique when set (partial unique index).
thread_external_id	text	Gmail thread ID; groups messages in one thread.
sentiment	interaction_sentiment enum	positive, neutral, negative, unknown. Optional. AI-assigned on Gmail ingest.
next_action_set	boolean	True if this interaction resulted in setting a new next_action on the opportunity. UI hint.
attachments	jsonb	Array of { name, url, size, type } — references to files (Google Drive links or Supabase Storage paths).
logged_by	uuid FK → team_members	Rep who logged (or system user for auto-ingest).
created_at	timestamptz	Default now().

3.1.3 Enum definitions

Five new enums needed; one existing enum extended. All in migration 119.

opportunity_stage

```
CREATE TYPE opportunity_stage AS ENUM (
  'Discovery',     -- Rep identified client; gathering intel
  'Qualifying',    -- Confirmed interest; assessing fit + budget
  'Proposal',      -- Proposal built, not yet sent or under review
  'Negotiation',   -- Proposal sent; edits/negotiation active
  'Won',           -- Proposal signed, contract created
  'Lost'           -- Abandoned, declined, or timed out
);
```

opportunity_channel

```
CREATE TYPE opportunity_channel AS ENUM (
  'inbound_call',  'outbound_call', 'email',     'meeting',
  'referral',      'walk_in',       'web_form',  'event',
  'renewal',       'other'
```

```
);
```

opportunity_outcome

```
CREATE TYPE opportunity_outcome AS ENUM (
  'qualified_not_ready',  -- Good fit, wrong timing
  'budget',               -- Priced out
  'competitor',           -- Chose competing outlet
  'no_response',          -- Ghost
  'timing',               -- Seasonal / event-dependent; revisit flag
  'product_fit',          -- We don't offer what they need
  'other'
);
```

interaction_type, direction, disposition, source, sentiment

```
CREATE TYPE interaction_type AS ENUM (
  'call', 'email', 'meeting', 'text', 'note', 'task', 'voicemail', 'dm'
);

CREATE TYPE interaction_direction AS ENUM ('inbound', 'outbound', 'internal');

CREATE TYPE interaction_disposition AS ENUM (
  'completed', 'no_answer', 'left_voicemail',
  'rescheduled', 'cancelled', 'no_show'
);

CREATE TYPE interaction_source AS ENUM (
  'manual', 'gmail_ingest', 'calendar_ingest', 'phone_log', 'text_ingest'
);

CREATE TYPE interaction_sentiment AS ENUM ('positive', 'neutral', 'negative',
'unknown');
```

3.1.4 Stage weighting (for forecast)

Each stage carries a weight used to compute weighted_value on the opportunity row. Stored as a constant in a view function, not in the enum, so publisher can tune without migration.

Discovery	10% · Exploratory; low commitment
Qualifying	25% · Interest confirmed
Proposal	50% · Proposal built or sent
Negotiation	75% · Under active review
Won	100% · Signed
Lost	0% · Excluded from pipeline

Exposed via stage_weight(opportunity_stage) stable SQL function. Weighted pipeline value on rep dashboard uses SUM(estimated_value * stage_weight(stage)) grouped by assigned_to.

3.2 Relationship to existing entities

opportunities ↔ sales (historical)

Historical sales rows in Closed status are NOT retroactively linked to opportunities. A nullable opportunity_id column will be added to sales for the new flow only. Views and reports that need to unify the two simply union them.

opportunities ↔ proposals

One-to-one nullable. A proposal can exist without an opportunity (edge case: rep skips the opportunity stage and builds a proposal directly for a warm client). Opportunity links to its proposal via proposal_id. When signature fires, opportunity.stage auto-flips to Won, contract_id is populated.

opportunities ↔ clients

Many-to-one. A single client can have multiple open opportunities (fall campaign + holiday special + special section). Client status (Active/Renewal/Lapsed/Lead) remains separate from opportunity stage — the client can be Active (has a contract) while simultaneously having three open Negotiation-stage opportunities.

interactions ↔ activity_log

activity_log stays for system events (contract_created, invoice_sent, commission_paid). interactions is for human communications. The two should not overlap. Client profile timeline UI merges both streams chronologically.

3.3 Gmail auto-ingest worker

An Edge Function polls Gmail via the Google Workspace connector every 5 minutes for each rep with Gmail authorized. For each new message, it attempts to match and create an interaction row.

3.3.1 Matching algorithm

Matching runs in this order; first match wins:

- **1. Thread continuation.** If an existing interaction has the same thread_external_id (Gmail thread ID), append the new message as a new interaction on the same opportunity.
- **2. Contact email exact match.** Look up client_contacts.email; if found, use that contact's client_id. Among open opportunities for that client (stage NOT IN (Won, Lost)), assign to the most recently touched one.
- **3. Domain match.** If no contact match, extract domain from the sender. Match to clients where any contact email shares the domain. If 1 client matches and 1 open opportunity exists → assign. If ambiguous, fall through.
- **4. Unmatched inbox.** If no match, create the interaction with opportunity_id = null, client_id = null, and flag in an "Unmatched Comms" tray on the rep's dashboard. Rep assigns manually; the system learns the contact email for next time (adds to client_contacts on confirm).

3.3.2 Body processing

- Strip quoted replies (previous message threading) — keep only the new content of this message.
- Strip HTML; preserve text with line breaks.
- Cap body storage at 8KB — truncate and store full URL to Gmail thread for the rep to open.
- Extract attachment metadata (name, size, MIME type) into interactions.attachments; do not copy attachment contents.
- Run lightweight sentiment classifier (positive / neutral / negative) via existing AI integration; store in interactions.sentiment.

3.3.3 Outbound ingestion

When a rep sends an email from Gmail, Gmail Push notifications fire the same worker. The direction is inferred from sender: if sender matches a team_members.email, direction = outbound; otherwise inbound.

3.3.4 Privacy + compliance

- Only the rep's own inbox is ingested, scoped to their OAuth token.
- Emails to/from non-client domains (team@13starsmedia.com, personal addresses) are ignored — only messages that match a client contact or domain.
- Each rep has a setting (team_members.gmail_ingest_enabled) to opt out entirely.
- Ingested interactions are flagged source = gmail_ingest and visible only to the rep who owns the opportunity + publisher role.

3.4 RPCs

3.4.1 create_opportunity

```
CREATE OR REPLACE FUNCTION create_opportunity(
  p_client_id uuid,
  p_name text,
  p_source_channel opportunity_channel,
  p_assigned_to uuid,
  p_estimated_value numeric DEFAULT NULL,
  p_expected_close_date date DEFAULT NULL,
  p_publications text[] DEFAULT NULL,
  p_notes text DEFAULT NULL
) RETURNS uuid
LANGUAGE plpgsql
SECURITY DEFINER
```

```
AS $$
DECLARE v_id uuid;
BEGIN
  INSERT INTO opportunities (
    client_id, name, stage, source_channel, assigned_to,
    estimated_value, weighted_value, expected_close_date,
    publications_of_interest, notes, created_by
  ) VALUES (
    p_client_id, p_name, 'Discovery', p_source_channel, p_assigned_to,
    p_estimated_value, COALESCE(p_estimated_value * 0.10, 0), p_expected_close_date,
    p_publications, p_notes, auth.uid()
  ) RETURNING id INTO v_id;

  INSERT INTO interactions (opportunity_id, client_id, type, direction, occurred_at,
subject, body, logged_by, source)
  VALUES (v_id, p_client_id, 'note', 'internal', now(),
    'Opportunity created', p_notes, auth.uid(), 'manual');

  RETURN v_id;
END;
$$;
```

3.4.2 advance_opportunity_stage

Single-entry mutator for stage transitions. Enforces legal transitions, stamps actual_close_date on Won/Lost, cascades status to linked proposal.

```
CREATE OR REPLACE FUNCTION advance_opportunity_stage(
  p_opportunity_id uuid,
  p_new_stage opportunity_stage,
  p_outcome opportunity_outcome DEFAULT NULL,
  p_lost_reason_detail text DEFAULT NULL
) RETURNS opportunities
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE v_opp opportunities%ROWTYPE;
BEGIN
  SELECT * INTO v_opp FROM opportunities WHERE id = p_opportunity_id FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'Opportunity % not found', p_opportunity_id; END
IF;

  IF p_new_stage = 'Lost' AND p_outcome IS NULL THEN
    RAISE EXCEPTION 'outcome is required when stage = Lost';
  END IF;

  UPDATE opportunities SET
    stage = p_new_stage,
    outcome = p_outcome,
    lost_reason_detail = p_lost_reason_detail,
```

```
    actual_close_date = CASE WHEN p_new_stage IN ('Won','Lost') THEN CURRENT_DATE
ELSE NULL END,
    weighted_value = estimated_value * stage_weight(p_new_stage),
    updated_at = now()
  WHERE id = p_opportunity_id
  RETURNING * INTO v_opp;

  INSERT INTO interactions (opportunity_id, client_id, type, direction, occurred_at,
subject, logged_by, source)
  VALUES (p_opportunity_id, v_opp.client_id, 'note', 'internal', now(),
    'Stage: ' || p_new_stage::text, auth.uid(), 'manual');

  RETURN v_opp;
END;
$$;
```

3.4.3 log_interaction

Universal write-path for manually logged interactions. Updates opportunity.last_interaction_at via trigger.

```
CREATE OR REPLACE FUNCTION log_interaction(
  p_opportunity_id uuid,
  p_type interaction_type,
  p_direction interaction_direction,
  p_occurred_at timestamptz,
  p_subject text DEFAULT NULL,
  p_body text DEFAULT NULL,
  p_contact_id uuid DEFAULT NULL,
  p_duration_minutes integer DEFAULT NULL,
  p_disposition interaction_disposition DEFAULT NULL
) RETURNS uuid
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE v_id uuid; v_client uuid;
BEGIN
  SELECT client_id INTO v_client FROM opportunities WHERE id = p_opportunity_id;
  IF v_client IS NULL THEN RAISE EXCEPTION 'Opportunity not found'; END IF;

  INSERT INTO interactions (
    opportunity_id, client_id, type, direction, occurred_at,
    subject, body, contact_id, duration_minutes, disposition,
    logged_by, source
  ) VALUES (
    p_opportunity_id, v_client, p_type, p_direction, p_occurred_at,
    p_subject, p_body, p_contact_id, p_duration_minutes, p_disposition,
    auth.uid(), 'manual'
  ) RETURNING id INTO v_id;
```

```
  RETURN v_id;
END;
$$;
```

3.4.4 ingest_gmail_interaction

Called only by the ingest Edge Function with a service-role key. Handles the matching cascade described in 3.3.1.

```
CREATE OR REPLACE FUNCTION ingest_gmail_interaction(
  p_external_id text,           -- Gmail message ID
  p_thread_external_id text,    -- Gmail thread ID
  p_from_email text,
  p_to_emails text[],
  p_subject text,
  p_body text,
  p_occurred_at timestamptz,
  p_rep_team_id uuid,           -- rep whose inbox this came from
  p_sentiment interaction_sentiment DEFAULT NULL,
  p_attachments jsonb DEFAULT NULL
) RETURNS jsonb  -- { interaction_id, opportunity_id, client_id, match_method,
ambiguous }
-- Body implements the 4-step matching algorithm.
-- Creates interaction with opportunity_id set if matched, null + unmatched flag
otherwise.
```

3.5 Triggers

tr_opportunity_last_interaction

After insert on interactions, bump opportunities.last_interaction_at if opportunity_id is set. Drives dashboard "stale opportunity" alerts.

tr_opportunity_weighted_value

Before update on opportunities when stage or estimated_value changes, recompute weighted_value.

tr_proposal_signed_wins_opportunity

On proposal_signatures insert with signed = true, if the proposal has an opportunity_id, call advance_opportunity_stage(opp_id, 'Won'). This closes the loop from Phase 1 to Phase 2.

3.6 UI requirements

3.6.1 Opportunity list view (tab on Sales Rep dashboard)

Kanban column layout. One column per stage (Discovery, Qualifying, Proposal, Negotiation). Won and Lost are collapsed to counts with a "show" toggle.

- Card shows: client name, opportunity name, estimated value, days since last interaction, priority indicator, assigned-to avatar.
- Card left-border color by staleness: green if last_interaction_at < 7 days, amber 7-14, red 14+.
- Drag-drop between columns calls advance_opportunity_stage. Drop to Lost triggers modal for outcome + detail.
- Filter bar: by rep (publisher only), by publication interest, by expected close window, by source channel.
- Search: client name, opportunity name, tags, notes full-text.
- "+ New Opportunity" button opens modal — client picker (with type-ahead), name, source channel, estimated value, expected close, notes.

3.6.2 Opportunity detail view

Single-page view with three columns on wide screens, stacked on mobile:

- **Left column — metadata.** Client link, stage, rep, values, dates, publications of interest, tags, priority. Inline-editable.
- **Center column — interaction timeline.** Reverse-chron stream of interactions. Each card: type icon, direction arrow, contact name, timestamp, subject, excerpt with "view full" expand. Compose box at top for quick add (call, email, note). "+ Log interaction" opens full modal.
- **Right column — actions.** Next action picker + date (writes to opportunity.next_action_* via a helper RPC, syncs to Calendar if enabled). Stage advance buttons. "Build Proposal from Opportunity" button pre-fills proposal with client + publications + products. "Archive as Lost" with outcome picker.

3.6.3 Unmatched comms tray

Surfaced as a banner on the rep's dashboard when Gmail ingest returns no match. Shows: sender email, snippet, timestamp. Rep clicks → modal with client picker; on confirm, the system updates client_contacts with the new email for future auto-match.

3.6.4 Client profile integration

Existing client profile gets a new "Opportunities" tab showing all opportunities for that client (open + closed). Interactions tab shows all interactions regardless of opportunity, with chips for which opportunity each belongs to.

3.7 Migration 119 — opportunities + interactions schema

```
-- 119_opportunity_interaction_model.sql

-- 1. Enum types
```

```
CREATE TYPE opportunity_stage AS ENUM (
  'Discovery','Qualifying','Proposal','Negotiation','Won','Lost'
);
CREATE TYPE opportunity_channel AS ENUM (
  'inbound_call','outbound_call','email','meeting','referral',
  'walk_in','web_form','event','renewal','other'
);
CREATE TYPE opportunity_outcome AS ENUM (
  'qualified_not_ready','budget','competitor','no_response',
  'timing','product_fit','other'
);
CREATE TYPE interaction_type AS ENUM (
  'call','email','meeting','text','note','task','voicemail','dm'
);
CREATE TYPE interaction_direction AS ENUM ('inbound','outbound','internal');
CREATE TYPE interaction_disposition AS ENUM (
  'completed','no_answer','left_voicemail','rescheduled','cancelled','no_show'
);
CREATE TYPE interaction_source AS ENUM (
  'manual','gmail_ingest','calendar_ingest','phone_log','text_ingest'
);
CREATE TYPE interaction_sentiment AS ENUM
('positive','neutral','negative','unknown');

-- 2. Tables
CREATE TABLE opportunities (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id uuid NOT NULL REFERENCES clients(id) ON DELETE RESTRICT,
  name text NOT NULL,
  stage opportunity_stage NOT NULL DEFAULT 'Discovery',
  source_channel opportunity_channel NOT NULL,
  outcome opportunity_outcome,
  assigned_to uuid NOT NULL REFERENCES team_members(id),
  estimated_value numeric(10,2),
  weighted_value numeric(10,2),
  expected_close_date date,
  actual_close_date date,
  publications_of_interest text[],
  products_of_interest text[],
  competitor_client_id uuid REFERENCES clients(id) ON DELETE SET NULL,
  proposal_id uuid REFERENCES proposals(id) ON DELETE SET NULL,
  contract_id uuid REFERENCES contracts(id) ON DELETE SET NULL,
  lost_reason_detail text,
  priority smallint DEFAULT 3 CHECK (priority BETWEEN 1 AND 5),
  tags text[],
  notes text,
  last_interaction_at timestamptz,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now(),
```

```
  created_by uuid REFERENCES team_members(id),
  CHECK (stage != 'Lost' OR outcome IS NOT NULL)
);

CREATE INDEX opp_client ON opportunities(client_id);
CREATE INDEX opp_assigned ON opportunities(assigned_to);
CREATE INDEX opp_stage ON opportunities(stage);
CREATE INDEX opp_open ON opportunities(assigned_to, stage) WHERE stage NOT IN
('Won','Lost');
CREATE INDEX opp_expected_close ON opportunities(expected_close_date);

CREATE TABLE interactions (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  opportunity_id uuid REFERENCES opportunities(id) ON DELETE CASCADE,
  client_id uuid NOT NULL REFERENCES clients(id) ON DELETE CASCADE,
  type interaction_type NOT NULL,
  direction interaction_direction NOT NULL,
  occurred_at timestamptz NOT NULL,
  duration_minutes integer,
  subject text,
  body text,
  contact_id uuid REFERENCES client_contacts(id) ON DELETE SET NULL,
  contact_email text,
  participant_ids uuid[],
  disposition interaction_disposition,
  source interaction_source NOT NULL DEFAULT 'manual',
  external_id text,
  thread_external_id text,
  sentiment interaction_sentiment,
  next_action_set boolean DEFAULT false,
  attachments jsonb,
  logged_by uuid REFERENCES team_members(id),
  created_at timestamptz DEFAULT now()
);

CREATE UNIQUE INDEX intx_external ON interactions(source, external_id) WHERE
external_id IS NOT NULL;
CREATE INDEX intx_opp ON interactions(opportunity_id, occurred_at DESC);
CREATE INDEX intx_client ON interactions(client_id, occurred_at DESC);
CREATE INDEX intx_thread ON interactions(thread_external_id) WHERE
thread_external_id IS NOT NULL;
CREATE INDEX intx_unmatched ON interactions(created_at)
  WHERE opportunity_id IS NULL AND client_id IS NULL;

-- 3. FK back-reference: sales.opportunity_id
ALTER TABLE sales ADD COLUMN opportunity_id uuid REFERENCES opportunities(id) ON
DELETE SET NULL;
CREATE INDEX sales_opp ON sales(opportunity_id) WHERE opportunity_id IS NOT NULL;
```

```
-- 4. Helper function
CREATE OR REPLACE FUNCTION stage_weight(s opportunity_stage) RETURNS numeric
LANGUAGE sql IMMUTABLE AS $$
  SELECT CASE s
    WHEN 'Discovery' THEN 0.10 WHEN 'Qualifying' THEN 0.25
    WHEN 'Proposal'  THEN 0.50 WHEN 'Negotiation' THEN 0.75
    WHEN 'Won'       THEN 1.00 WHEN 'Lost'        THEN 0.00
  END;
$$;

-- 5. RLS
ALTER TABLE opportunities ENABLE ROW LEVEL SECURITY;
ALTER TABLE interactions ENABLE ROW LEVEL SECURITY;

-- Policies: reps see their own + publisher sees all. (Full policies in Section 9.)
```

3.8 Acceptance criteria

- Dana can create an opportunity from a client profile in three clicks.
- Every email from Dana's Gmail to a known client contact appears as an interaction within 10 minutes, correctly linked to the most recent open opportunity.
- Opportunity kanban board renders in under 1 second with 500 open opportunities across 6 reps.
- Signing a proposal flips its parent opportunity to Won automatically, with actual_close_date stamped.
- Dashboard forecast (sum of weighted_value by rep) reconciles to within $1 of the source data.
- Unmatched comms tray never exceeds 50 items for a rep; the rep's "confirm match" action updates client_contacts so future messages auto-match.

4. Gap 2 — Press-to-Invoice Trigger

When the publisher marks an issue sent to press, every draft invoice for that issue needs to flip to sent, queue for QuickBooks sync, and queue for Stripe auto-charge where a card is on file. Small surface area, highest operational leverage in the spec: this is what turns the proposal-to-contract conversion from a paper process into billed revenue.

Why this matters more than it looks

Today the system creates per-issue invoices in draft state during proposal conversion, but nothing moves them to sent. This means Cami has to manually fire invoices issue-by-issue — the one thing the Newspaper Manager replacement was explicitly meant to automate.

The freeze_on_press trigger already exists on issues.sent_to_press_at (it freezes allocations and stamps thread expiry). We extend that pattern to invoices.

4.1 Behavior specification

The trigger fires when issues.sent_to_press_at transitions from NULL to a timestamp. For every invoice where the invoice has at least one invoice_line referencing this issue_id:

- **1. Flip to sent.** Update invoices.status from draft to sent. Stamp invoices.issue_date = today. Compute due_date from client payment_terms (default 30 days).
- **2. Lock the invoice.** Set locked_at = now(). No more line-item edits after press.
- **3. Enqueue QuickBooks sync.** Insert a row into integration_jobs with type = qbo_invoice_create, payload = { invoice_id }.
- **4. Enqueue Stripe auto-charge.** If client has stripe_payment_method_id, insert integration_jobs row with type = stripe_auto_charge. Only if credit_hold is false.
- **5. Stamp initial reminder cadence.** Pre-schedule reminders — no immediate send, but compute the dates and store as invoice_reminders rows (new table, Section 4.3).
- **6. Log activity.** Insert activity_log entry "Invoice sent on press" per invoice. Lets the Office Admin see exactly what fired when.

4.2 Edge cases

Mixed-issue invoices

An invoice may have lines from multiple issues (rare, but possible for custom billing). Rule: the invoice flips to sent when ANY of its issues is sent to press. A separate column partial_press_sent tracks the mixed state so subsequent press events don't re-fire the Stripe charge.

Re-press scenarios

If sent_to_press_at is cleared (publisher un-marks, rare) and then re-set, the trigger SHOULD NOT re-fire on invoices already in sent or paid status. The trigger checks invoices.status before flipping.

Credit-held clients

If client.credit_hold = true at press time, the invoice still flips to sent (visible to client), but the Stripe auto-charge is skipped and a flag is set on the invoice: suppressed_for_credit_hold = true. Office Admin resolves the hold, then can trigger the charge manually.

Missing Stripe card

If client has no stripe_payment_method_id, the auto-charge is skipped. Invoice still flips to sent and enters the reminder cadence. No error, no retry queue — the rep or Cami will follow up.

Draft invoice missing lines

If an invoice somehow has no invoice_lines referencing the pressed issue (bug or manual edit), the trigger does not touch it. A row is logged in integration_jobs_errors with reason = no_matching_lines so the Office Admin can see the anomaly.

4.3 New table: invoice_reminders

Pre-scheduled reminder jobs, one row per planned send. The scheduler polls this table instead of computing dates at send time. Allows Office Admin to see what's queued and cancel reminders (e.g., after a phone call commitment to pay).

Column	Type	Purpose
`id`	`uuid PK`	Primary key.
`invoice_id`	`uuid FK → invoices`	NOT NULL. ON DELETE CASCADE.
`cadence`	`reminder_cadence enum`	first, second, final. NOT NULL.
`scheduled_for`	`timestamptz`	When to send. NOT NULL.
`status`	`reminder_status enum`	pending, sent, cancelled, failed. Default pending.
`sent_at`	`timestamptz`	Stamped on send.
`cancelled_reason`	`text`	If cancelled (payment received, manual cancel).
`error_message`	`text`	If failed.
`template_version`	`text`	Which email template was used (for A/B testing later).
`created_at`	`timestamptz`	Default now().

Reminder schedule

Stored in org_settings.reminder_schedule (jsonb) so publisher can adjust. Default:

first	Due date + 3 days
second	Due date + 14 days (status → overdue at this point)
final	Due date + 28 days (triggers credit hold flag for Office Admin review)

4.4 Integration jobs queue

4.4.1 integration_jobs table

Generic async-work queue. Also used by Gmail ingest worker and QBO sync. Postgres-based queue is sufficient at this scale (~50k invoices per year, ~1500 per month at peak). Processing via Supabase Edge Functions polling on a cron.

Column	Type	Purpose
`id`	`uuid PK`	Primary key.
`type`	`integration_job_type enum`	qbo_invoice_create, qbo_invoice_update, qbo_payment_create, stripe_auto_charge, stripe_refund, gmail_ingest_poll, calendar_sync. NOT NULL.
`payload`	`jsonb`	Type-specific arguments. Documented per job type in an adjacent JSONB schema doc.
`status`	`job_status enum`	queued, running, succeeded, failed, dead. Default queued.
`attempts`	`integer`	Default 0. Max 5 before dead letter.
`last_attempt_at`	`timestamptz`	
`next_attempt_at`	`timestamptz`	Default now(); exponential backoff on failure.
`error_message`	`text`	
`result`	`jsonb`	Stored result on success (QBO id, Stripe charge id, etc.).
`locked_by`	`text`	Worker identifier; prevents double-processing.
`locked_until`	`timestamptz`	Release time for stuck jobs.
`priority`	`smallint`	Default 5. Lower = higher priority.
`related_entity_type`	`text`	invoice, payment, interaction, etc. For observability.
`related_entity_id`	`text`	FK-like but loose (entity types have different PK types).
`created_at`	`timestamptz`	Default now().

4.4.2 Worker pattern

One Edge Function per job type category, invoked by cron every minute. Each worker:

- Claims N jobs via an atomic UPDATE (locked_by, locked_until) on queued jobs.
- Processes each; on success, stamps succeeded + result.
- On failure, increments attempts, stamps next_attempt_at = now() + 2^attempts minutes.

- After 5 attempts, status = dead. Publisher has a "Dead jobs" screen to inspect + retry or dismiss.

4.5 Migration 120 — press-to-invoice trigger + queue

```
-- 120_press_to_invoice_trigger.sql

-- 1. Reminder scheduling table
CREATE TYPE reminder_cadence AS ENUM ('first','second','final');
CREATE TYPE reminder_status AS ENUM ('pending','sent','cancelled','failed');

CREATE TABLE invoice_reminders (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id uuid NOT NULL REFERENCES invoices(id) ON DELETE CASCADE,
  cadence reminder_cadence NOT NULL,
  scheduled_for timestamptz NOT NULL,
  status reminder_status NOT NULL DEFAULT 'pending',
  sent_at timestamptz,
  cancelled_reason text,
  error_message text,
  template_version text,
  created_at timestamptz DEFAULT now(),
  UNIQUE (invoice_id, cadence)
);
CREATE INDEX inv_rem_due ON invoice_reminders(scheduled_for) WHERE status =
'pending';

-- 2. Integration jobs queue
CREATE TYPE integration_job_type AS ENUM (
  'qbo_invoice_create','qbo_invoice_update','qbo_payment_create',
  'stripe_auto_charge','stripe_refund','gmail_ingest_poll','calendar_sync'
);
CREATE TYPE job_status AS ENUM ('queued','running','succeeded','failed','dead');

CREATE TABLE integration_jobs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  type integration_job_type NOT NULL,
  payload jsonb NOT NULL DEFAULT '{}'::jsonb,
  status job_status NOT NULL DEFAULT 'queued',
  attempts integer NOT NULL DEFAULT 0,
  last_attempt_at timestamptz,
  next_attempt_at timestamptz DEFAULT now(),
  error_message text,
  result jsonb,
  locked_by text,
  locked_until timestamptz,
  priority smallint DEFAULT 5,
  related_entity_type text,
```

```
  related_entity_id text,
  created_at timestamptz DEFAULT now()
);
CREATE INDEX ij_ready ON integration_jobs(type, next_attempt_at, priority) WHERE
status = 'queued';
CREATE INDEX ij_running ON integration_jobs(locked_until) WHERE status = 'running';

-- 3. Add suppressed_for_credit_hold + partial_press_sent flags to invoices
ALTER TABLE invoices ADD COLUMN IF NOT EXISTS suppressed_for_credit_hold boolean
DEFAULT false;
ALTER TABLE invoices ADD COLUMN IF NOT EXISTS partial_press_sent boolean DEFAULT
false;

-- 4. Function: fire invoice cascade on press
CREATE OR REPLACE FUNCTION fire_invoice_cascade_on_press(p_issue_id text)
RETURNS integer
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_reminder_cfg jsonb;
  v_first_days int;
  v_second_days int;
  v_final_days int;
  v_count int := 0;
  v_inv record;
  v_client record;
  v_due_days int;
BEGIN
  SELECT COALESCE(billing_config->'reminder_schedule',
    '{"first":3,"second":14,"final":28}'::jsonb)
    INTO v_reminder_cfg FROM org_settings LIMIT 1;
  v_first_days  := (v_reminder_cfg->>'first')::int;
  v_second_days := (v_reminder_cfg->>'second')::int;
  v_final_days  := (v_reminder_cfg->>'final')::int;

  FOR v_inv IN
    SELECT DISTINCT i.id, i.client_id
    FROM invoices i
    JOIN invoice_lines il ON il.invoice_id = i.id
    WHERE il.issue_id = p_issue_id
      AND i.status = 'draft'
  LOOP
    SELECT credit_hold, stripe_payment_method_id, COALESCE(
      (SELECT (payment_terms)::int FROM contracts c
       WHERE c.id = (SELECT contract_id FROM invoices WHERE id = v_inv.id)),
      30) AS due_days
    INTO v_client FROM clients WHERE id = v_inv.client_id;
```

```
    UPDATE invoices SET
      status = 'sent',
      issue_date = CURRENT_DATE,
      due_date = CURRENT_DATE + (COALESCE(v_client.due_days, 30) || '
days')::interval,
      locked_at = now(),
      suppressed_for_credit_hold = COALESCE(v_client.credit_hold, false),
      updated_at = now()
    WHERE id = v_inv.id;

    -- Queue QBO sync
    INSERT INTO integration_jobs (type, payload, related_entity_type,
related_entity_id)
    VALUES ('qbo_invoice_create', jsonb_build_object('invoice_id', v_inv.id),
            'invoice', v_inv.id::text);

    -- Queue Stripe charge if card on file + no credit hold
    IF v_client.stripe_payment_method_id IS NOT NULL
       AND NOT COALESCE(v_client.credit_hold, false) THEN
      INSERT INTO integration_jobs (type, payload, related_entity_type,
related_entity_id, next_attempt_at)
      VALUES ('stripe_auto_charge', jsonb_build_object('invoice_id', v_inv.id),
              'invoice', v_inv.id::text, now() + interval '5 minutes');
    END IF;

    -- Schedule reminder cadence
    INSERT INTO invoice_reminders (invoice_id, cadence, scheduled_for) VALUES
      (v_inv.id, 'first',  (CURRENT_DATE + (v_client.due_days + v_first_days)  || '
days')::interval + CURRENT_DATE),
      (v_inv.id, 'second', (CURRENT_DATE + (v_client.due_days + v_second_days) || '
days')::interval + CURRENT_DATE),
      (v_inv.id, 'final',  (CURRENT_DATE + (v_client.due_days + v_final_days)  || '
days')::interval + CURRENT_DATE)
    ON CONFLICT (invoice_id, cadence) DO NOTHING;

    -- Log activity
    INSERT INTO activity_log (type, client_id, detail)
    VALUES ('invoice_sent_on_press',
            v_inv.client_id,
            'Invoice ' || (SELECT invoice_number FROM invoices WHERE id = v_inv.id)
||
            ' sent on press of issue ' || p_issue_id);

    v_count := v_count + 1;
  END LOOP;
  RETURN v_count;
END;
$$;
```

```
-- 5. Trigger wiring
CREATE OR REPLACE FUNCTION tf_press_fires_invoices()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
  IF NEW.sent_to_press_at IS NOT NULL
     AND (OLD.sent_to_press_at IS NULL OR OLD.sent_to_press_at IS DISTINCT FROM
NEW.sent_to_press_at) THEN
    PERFORM fire_invoice_cascade_on_press(NEW.id);
  END IF;
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS tr_press_fires_invoices ON issues;
CREATE TRIGGER tr_press_fires_invoices
  AFTER UPDATE OF sent_to_press_at ON issues
  FOR EACH ROW
  EXECUTE FUNCTION tf_press_fires_invoices();
```

4.6 Edge Function: press-to-invoice-worker

Even though the cascade runs in-trigger for DB consistency, the slower external calls (QBO + Stripe) run asynchronously via the queue. Worker function signatures:

```
supabase/functions/qbo-invoice-sync/index.ts
// Polls integration_jobs WHERE type IN (qbo_invoice_create, qbo_invoice_update).
// Claims 10 jobs, for each: fetches invoice + client + lines, calls QBO API, stores
quickbooks_id.

supabase/functions/stripe-auto-charge/index.ts
// Polls integration_jobs WHERE type = stripe_auto_charge.
// Claims 10 jobs, for each: creates PaymentIntent with stripe_customer_id +
payment_method,
// confirms charge, on success inserts payments row + calls
apply_payment_to_invoices.

supabase/functions/reminder-dispatcher/index.ts
// Polls invoice_reminders WHERE scheduled_for <= now() AND status = 'pending'.
// Claims batch, sends email via Gmail API or Resend, stamps sent_at + status =
sent.
// On final cadence, also flags client for credit hold review.
```

4.7 UI requirements

Press day confirmation modal

When the publisher clicks "Mark issue sent to press" on the issue schedule, a confirmation modal surfaces:

- Count of invoices that will be fired (preview query against the cascade logic).
- Total dollar value fired.
- Any invoices flagged for credit hold review (surfaced as warnings, will still fire but suppressed from Stripe).
- "Confirm — send invoices" button triggers the press timestamp.

Integration jobs dashboard (publisher/admin only)

Table view of recent jobs with filter by type + status. Dead jobs get a "Retry" button (resets attempts to 0, status to queued) and a "Dismiss" button (status to dead, never retried).

4.8 Acceptance criteria

- Publisher marks an issue sent to press; within 2 seconds, every draft invoice for that issue shows "Sent" status in the admin UI.
- Within 5 minutes, each invoice appears in QuickBooks with a quickbooks_id stored on the invoice row.
- Clients with saved cards receive a Stripe charge within 5 minutes; successful charges create payments rows; invoice flips to paid; client.total_spend updates.
- Reminder rows exist for every sent invoice at 3, 14, and 28 days past due date.
- Un-marking an issue (clearing sent_to_press_at) does NOT fire the cascade a second time if re-pressed.
- A failed Stripe charge (card declined) does not fail the cascade; invoice remains in sent status and enters reminder cadence normally.

5. Gap 3 — Renewal Automation

Renewal is the highest-retention, lowest-effort revenue in the business. Every advertiser with an expiring contract is a warm lead; the only question is whether the rep sees them in time. The current system buries them — clients with expired contracts flip to Lapsed automatically but by then the relationship has cooled. The fix is a 60-day leading window that flags, drafts, and surfaces the renewal before the rep has to remember.

What changes from current state

Client status enum gains no new value — Renewal already exists but is computed at 30 days. Widen to 60 days.

Renewal surfaces as a kanban column on the Sales Rep dashboard, driven by a view that computes window eligibility.

A renewal proposal generator pre-fills a new proposal from the expiring contract's lines, with stored rate changes applied.

An opportunity (from Gap 1) is automatically created in the Qualifying stage with source_channel = renewal when the status flips.

5.1 Renewal state machine

Three stages for a client with an expiring contract, each a computed state (not stored). Transitions happen as the clock moves forward; no manual actions required.

Active (T-60+ days)	Contract end date more than 60 days out. Client status = Active. No renewal surfacing.
Renewal (T-60 to T-0)	Contract end within 60 days, still before end date. Client status = Renewal. Dashboard surfaces with amber indicator. Auto-opportunity created at T-60.
Urgent Renewal (T-30 to T-0)	Contract end within 30 days. Same Renewal status, but dashboard shows red indicator. Renewal proposal auto-drafted at T-30 if none exists yet.
Lapsed (T+1+ days)	Past end date, no renewal signed. Status flips to Lapsed. Still surfaced on dashboard under "Recently lapsed" for 45 days.
Dormant (T+46+ days)	Past end date 45+ days, no outreach. Still Lapsed status but falls off primary dashboard; appears only in the outreach campaign tool.

5.2 Status computation update

The current client status trigger/function uses 30 days as the Renewal boundary. Update to 60 days. The function should read these constants from org_settings so the publisher can tune per business cycle.

New org_settings fields

Column	Type	Purpose
`renewal_window_days`	`integer`	Default 60. Days before contract end_date when status flips to Renewal.
`urgent_renewal_days`	`integer`	Default 30. Days before end_date when renewal is flagged urgent.
`lapsed_outreach_days`	`integer`	Default 45. Days past end_date when a Lapsed client falls off primary dashboard.
`auto_draft_renewal_at_days`	`integer`	Default 30. Days before end_date when AI draft auto-generates.

Updated recompute_client_status()

Extend the existing function. Logic:

```
CREATE OR REPLACE FUNCTION recompute_client_status(p_client_id uuid)
RETURNS client_status
LANGUAGE plpgsql
AS $$
DECLARE
  v_renewal_days int;
  v_today date := CURRENT_DATE;
  v_contract_end date;
  v_has_closed_sale boolean;
  v_has_future_ad boolean;
  v_status client_status;
BEGIN
  SELECT COALESCE(renewal_window_days, 60) INTO v_renewal_days FROM org_settings
LIMIT 1;

  SELECT MAX(c.end_date), EXISTS(SELECT 1 FROM sales WHERE client_id = p_client_id
AND status = 'Closed')
  INTO v_contract_end, v_has_closed_sale
  FROM contracts c WHERE c.client_id = p_client_id;

  SELECT EXISTS(
    SELECT 1 FROM sales s
    JOIN issues i ON s.issue_id = i.id
    WHERE s.client_id = p_client_id AND s.status = 'Closed'
      AND i.date > v_today + interval '30 days'
  ) INTO v_has_future_ad;

  IF NOT v_has_closed_sale THEN
    v_status := 'Lead';
  ELSIF v_has_future_ad
        AND (v_contract_end IS NULL OR v_contract_end > v_today + (v_renewal_days ||
' days')::interval) THEN
    v_status := 'Active';
```

```
  ELSIF v_contract_end IS NOT NULL
        AND v_contract_end >= v_today
        AND v_contract_end <= v_today + (v_renewal_days || ' days')::interval THEN
    v_status := 'Renewal';
  ELSIF v_contract_end IS NOT NULL AND v_contract_end < v_today THEN
    v_status := 'Lapsed';
  ELSE
    v_status := 'Lapsed';
  END IF;

  UPDATE clients SET status = v_status, updated_at = now() WHERE id = p_client_id;
  RETURN v_status;
END;
$$;
```

5.3 Nightly renewal job

A cron-invoked Edge Function runs once per day at 2am Pacific, computing the three renewal effects. All work is idempotent — re-running the job does not create duplicates.

Job steps

- **Step 1 — Recompute statuses.** For every client with a contract end_date in the next 90 days OR in the past 60 days, call recompute_client_status. Use a batched SELECT to find the candidates in one query.
- **Step 2 — Auto-create renewal opportunities.** For every client newly flipped to Renewal status (determined by checking activity_log for status changes in the last 24 hours) that does NOT already have an open opportunity, call create_opportunity with source_channel = renewal, name = "Renewal: [Client name] [Year]", publications = from expiring contract.
- **Step 3 — Auto-draft renewal proposals.** For every client within the urgent_renewal_days window where the renewal opportunity exists but no linked proposal yet, call draft_renewal_proposal (Section 5.4). Links the resulting proposal to the opportunity.
- **Step 4 — Surface on dashboard.** No explicit step — the dashboard view queries live. But the job stamps an org_settings.last_renewal_run timestamp so the dashboard can show "last refreshed X minutes ago."

5.4 Renewal proposal generator

The draft_renewal_proposal RPC copies the expiring contract's lines into a new proposal, applies rate adjustments, and stamps it as an AI draft. The rep reviews, edits, and sends — no manual line-by-line rebuild.

5.4.1 Rate adjustment rules

Adjustments live in org_settings.renewal_pricing_rules (jsonb). Default:

Default annual increase	5% over previous rate
Multi-year loyalty discount	-2% per consecutive renewal year (max 10 years)
New rate card override	If current publication rate_card has a higher published rate than contract rate × 1.05, use the rate card instead
Inflation floor	Never decrease rate from prior contract unless publisher overrides

5.4.2 draft_renewal_proposal RPC

```
CREATE OR REPLACE FUNCTION draft_renewal_proposal(
  p_opportunity_id uuid
) RETURNS uuid
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_opp opportunities%ROWTYPE;
  v_prior_contract contracts%ROWTYPE;
  v_proposal_id uuid;
  v_rate_increase numeric := 0.05;
  v_line record;
  v_new_rate numeric;
  v_consecutive_years int;
  v_loyalty_discount numeric := 0;
BEGIN
  SELECT * INTO v_opp FROM opportunities WHERE id = p_opportunity_id;
  IF v_opp.stage IN ('Won','Lost') THEN
    RAISE EXCEPTION 'Opportunity already closed';
  END IF;

  SELECT c.* INTO v_prior_contract
  FROM contracts c
  WHERE c.client_id = v_opp.client_id
    AND c.end_date IS NOT NULL
  ORDER BY c.end_date DESC LIMIT 1;

  IF v_prior_contract.id IS NULL THEN
    RAISE EXCEPTION 'No prior contract to base renewal on';
  END IF;

  SELECT COUNT(DISTINCT DATE_PART('year', start_date))
  INTO v_consecutive_years
  FROM contracts WHERE client_id = v_opp.client_id;
  v_loyalty_discount := LEAST(v_consecutive_years * 0.02, 0.10);
```

```
  INSERT INTO proposals (client_id, name, status, term_months, date, assigned_to,
art_source, created_by, renewal_date)
  VALUES (
    v_opp.client_id,
    'Renewal: ' || (SELECT name FROM clients WHERE id = v_opp.client_id) || ' ' ||
DATE_PART('year', CURRENT_DATE + interval '60 days'),
    'Draft',
    COALESCE(v_prior_contract.end_date - v_prior_contract.start_date, 365) / 30,
    CURRENT_DATE,
    v_opp.assigned_to,
    (SELECT last_art_source FROM clients WHERE id = v_opp.client_id),
    auth.uid(),
    v_prior_contract.end_date
  ) RETURNING id INTO v_proposal_id;

  FOR v_line IN
    SELECT * FROM contract_lines WHERE contract_id = v_prior_contract.id
  LOOP
    v_new_rate := v_line.rate * (1 + v_rate_increase - v_loyalty_discount);
    INSERT INTO proposal_lines (
      proposal_id, publication_id, ad_size, price, quantity,
      placement_id, page_guarantee, rh_guarantee, placement_guarantee_pct
    ) VALUES (
      v_proposal_id, v_line.publication_id, v_line.ad_size,
      v_new_rate, v_line.quantity,
      v_line.placement_id, v_line.page_guarantee,
      v_line.rh_guarantee, v_line.placement_guarantee_pct
    );
  END LOOP;

  UPDATE opportunities SET proposal_id = v_proposal_id WHERE id = p_opportunity_id;

  INSERT INTO interactions (opportunity_id, client_id, type, direction, occurred_at,
subject, body, source, logged_by)
  VALUES (p_opportunity_id, v_opp.client_id,
          'note', 'internal', now(),
          'Renewal proposal auto-drafted',
          'Based on contract ' || v_prior_contract.name || ', ' ||
          (v_rate_increase * 100)::text || '% increase, ' ||
          (v_loyalty_discount * 100)::text || '% loyalty discount.',
          'manual', auth.uid());

  RETURN v_proposal_id;
END;
$$;
```

5.5 Renewal queue view

A SQL view powers the Sales Rep dashboard's Renewals tab. Keeps the UI logic simple and performance predictable.

```
CREATE OR REPLACE VIEW v_renewal_queue AS
SELECT
  c.id AS client_id,
  c.name AS client_name,
  c.rep_id,
  c.contract_end_date,
  c.contract_end_date - CURRENT_DATE AS days_to_end,
  ctr.id AS expiring_contract_id,
  ctr.total_value AS expiring_contract_value,
  opp.id AS renewal_opportunity_id,
  opp.stage AS opportunity_stage,
  opp.proposal_id,
  prop.status AS proposal_status,
  CASE
    WHEN c.contract_end_date < CURRENT_DATE THEN 'lapsed'
    WHEN c.contract_end_date <= CURRENT_DATE + interval '30 days' THEN 'urgent'
    WHEN c.contract_end_date <= CURRENT_DATE + interval '60 days' THEN 'renewal'
    ELSE 'upcoming'
  END AS bucket
FROM clients c
JOIN contracts ctr ON ctr.client_id = c.id
  AND ctr.end_date = c.contract_end_date
  AND (ctr.status = 'active' OR ctr.status IS NULL)
LEFT JOIN opportunities opp ON opp.client_id = c.id AND opp.source_channel =
'renewal' AND opp.stage NOT IN ('Won','Lost')
LEFT JOIN proposals prop ON prop.id = opp.proposal_id
WHERE c.status IN ('Active', 'Renewal', 'Lapsed')
  AND c.contract_end_date BETWEEN CURRENT_DATE - interval '45 days'
                              AND CURRENT_DATE + interval '60 days';
```

5.6 UI requirements

5.6.1 Renewals tab (Sales Rep dashboard)

Three sub-tabs, one per bucket from the view: Urgent (<=30d, red), Renewal (31-60d, amber), Lapsed (past end, gray). Count badge on each. Default view = Urgent.

- Card layout: client name + contract end date + days countdown (large, colored).
- Secondary line: expiring contract value + previous year industry spend percentile.
- Tertiary: status of renewal opportunity + proposal.

- Primary action button: if no proposal drafted yet → "Draft renewal proposal" (calls draft_renewal_proposal). If drafted but not sent → "Review + send". If sent → "View signature status".
- Secondary actions: "Open opportunity" (→ opportunity detail), "Log interaction" (quick-add).
- Sort: default by days_to_end asc within each bucket.
- Filter: by publication, by rep (publisher only), by value range.

5.6.2 Contract detail — renewal launchpad

On the contract detail page, a new "Renewal" card surfaces when the contract is within 90 days of end. Shows: countdown, projected renewal value (after increase + discount), one-click "Generate renewal proposal."

5.7 Migration 121 — renewal automation

```
-- 121_renewal_automation.sql

-- 1. org_settings new columns
ALTER TABLE org_settings
  ADD COLUMN IF NOT EXISTS renewal_window_days integer DEFAULT 60,
  ADD COLUMN IF NOT EXISTS urgent_renewal_days integer DEFAULT 30,
  ADD COLUMN IF NOT EXISTS lapsed_outreach_days integer DEFAULT 45,
  ADD COLUMN IF NOT EXISTS auto_draft_renewal_at_days integer DEFAULT 30,
  ADD COLUMN IF NOT EXISTS renewal_pricing_rules jsonb DEFAULT
'{"rate_increase":0.05,"loyalty_discount_per_year":0.02,"max_loyalty_discount":0.10}
'::jsonb,
  ADD COLUMN IF NOT EXISTS last_renewal_run timestamptz;

-- 2. Updated recompute_client_status (see Section 5.2 body)
-- 3. draft_renewal_proposal RPC (see Section 5.4.2)
-- 4. v_renewal_queue view (see Section 5.5)

-- 5. Nightly renewal job entry point (called by Edge Function)
CREATE OR REPLACE FUNCTION run_nightly_renewal_job()
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_recomputed int := 0;
  v_opps_created int := 0;
  v_drafts_created int := 0;
  v_client record;
  v_opp_id uuid;
  v_prop_id uuid;
  v_auto_draft_days int;
```

```
BEGIN
  SELECT COALESCE(auto_draft_renewal_at_days, 30) INTO v_auto_draft_days FROM
org_settings LIMIT 1;

  FOR v_client IN
    SELECT id, status, contract_end_date, rep_id FROM clients
    WHERE contract_end_date IS NOT NULL
      AND contract_end_date BETWEEN CURRENT_DATE - interval '60 days'
                                AND CURRENT_DATE + interval '90 days'
  LOOP
    PERFORM recompute_client_status(v_client.id);
    v_recomputed := v_recomputed + 1;

    IF v_client.contract_end_date <= CURRENT_DATE + interval '60 days'
       AND v_client.contract_end_date >= CURRENT_DATE THEN
      -- Ensure renewal opportunity exists
      IF NOT EXISTS (
        SELECT 1 FROM opportunities
        WHERE client_id = v_client.id
          AND source_channel = 'renewal'
          AND stage NOT IN ('Won','Lost')
      ) THEN
        INSERT INTO opportunities (client_id, name, stage, source_channel,
assigned_to, estimated_value)
        VALUES (
          v_client.id,
          'Renewal: ' || (SELECT name FROM clients WHERE id = v_client.id) || ' ' ||
          DATE_PART('year', v_client.contract_end_date + interval '1 year'),
          'Qualifying',
          'renewal',
          v_client.rep_id,
          (SELECT total_value FROM contracts WHERE client_id = v_client.id ORDER BY
end_date DESC LIMIT 1)
        ) RETURNING id INTO v_opp_id;
        v_opps_created := v_opps_created + 1;
      END IF;
    END IF;

    IF v_client.contract_end_date <= CURRENT_DATE + (v_auto_draft_days || '
days')::interval
       AND v_client.contract_end_date >= CURRENT_DATE THEN
      SELECT id INTO v_opp_id FROM opportunities
      WHERE client_id = v_client.id
        AND source_channel = 'renewal'
        AND stage NOT IN ('Won','Lost')
        AND proposal_id IS NULL
      LIMIT 1;
      IF v_opp_id IS NOT NULL THEN
        BEGIN
```

```
          PERFORM draft_renewal_proposal(v_opp_id);
          v_drafts_created := v_drafts_created + 1;
        EXCEPTION WHEN OTHERS THEN
          -- Log but don't fail the whole job
          INSERT INTO activity_log (type, client_id, detail)
          VALUES ('renewal_draft_failed', v_client.id, SQLERRM);
        END;
      END IF;
    END IF;
  END LOOP;

  UPDATE org_settings SET last_renewal_run = now();

  RETURN jsonb_build_object(
    'recomputed', v_recomputed,
    'opportunities_created', v_opps_created,
    'drafts_created', v_drafts_created,
    'completed_at', now()
  );
END;
$$;
```

5.8 Acceptance criteria

- A client with contract_end_date 59 days from today appears in the Renewal bucket on the rep dashboard the morning after the nightly job runs.
- The same client has a renewal opportunity in Qualifying stage, source_channel = renewal, assigned to the correct rep.
- At 30 days from end, a Draft proposal exists linked to that opportunity, with lines copied from the expiring contract at 5% increased rates (or applied loyalty discount).
- Rep can one-click "Send renewal proposal" which fires the existing proposal send flow.
- If the client signs, the existing signature trigger flips the opportunity to Won and creates the new contract via convert_proposal_to_contract. No custom renewal flow.
- If the rep does not act and the contract end date passes, status flips to Lapsed, but the opportunity remains open for 45 more days in the Lapsed tab.

6. Gap 4 — Commission Payout Execution

The commission system is the most thoroughly built and least exercised part of the codebase. 35,825 ledger entries exist; 33,555 have reached earned status (both triggers fired); zero have ever been paid. The payout schema is in place; what is missing is the confident operational workflow for Hayley to click "pay" and have the money and the records match.

Why this is a gap even though schema exists

A commission system isn't working until money has moved through it. The current state means that when Dana or Christie actually earn commissions in production, Hayley will face the UI cold with no validated run to reference.

The fix is not more schema. It is: a pre-run reconciliation screen, a confirmation gate, an immutable payout record, and a post-run audit trail that reps can self-serve.

6.1 Payout lifecycle

Four states for a payout run. Each transition is operator-driven; no status flips automatically.

draft	Publisher has started composing a run. Can be abandoned without effect.
approved	Publisher has reviewed totals and clicked Approve. Ledger entries locked to this payout, but no external payment yet.
paid	Publisher has marked the approved run as paid externally (QuickBooks payroll, manual check, etc.). Immutable.
void	Published in error. Ledger entries released back to earned status. Historical record preserved.

6.2 RPC: compose_commission_payout

Single entry point for building a draft payout. Takes a rep ID and a date range, finds eligible ledger entries, computes totals, creates a commission_payouts row in draft status, and links the entries to it.

```
CREATE OR REPLACE FUNCTION compose_commission_payout(
  p_salesperson_id uuid,
  p_period_start date,
  p_period_end date,
  p_include_ledger_ids uuid[] DEFAULT NULL  -- optional: explicit selection override
) RETURNS jsonb  -- { payout_id, count, total, bonus_applied, base_total }
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_payout_id uuid;
  v_count int;
```

```
  v_base_total numeric := 0;
  v_bonus_applied numeric := 0;
  v_total numeric := 0;
  v_goal_attainment numeric;
  v_bonus_pct numeric := 0;
  v_period text;
BEGIN
  v_period := TO_CHAR(p_period_start, 'YYYY-MM') || '_' || TO_CHAR(p_period_end,
'YYYY-MM');

  -- Compute goal attainment for the period to determine bonus tier
  SELECT
    COALESCE(SUM(cl.sale_amount), 0) / NULLIF(SUM(cg.goal * cg.share_pct / 100), 0)
  INTO v_goal_attainment
  FROM commission_ledger cl
  LEFT JOIN commission_issue_goals cg ON cg.issue_id = cl.issue_id AND
cg.publication_id = cl.publication_id
  WHERE cl.salesperson_id = p_salesperson_id
    AND cl.earned_at BETWEEN p_period_start AND p_period_end
    AND cl.status = 'earned';

  v_bonus_pct := LEAST(GREATEST(FLOOR((COALESCE(v_goal_attainment, 0) - 1) * 10) *
0.02, 0), 0.10);

  -- Create draft payout
  INSERT INTO commission_payouts (salesperson_id, period, status, total_amount,
commission_count, notes)
  VALUES (p_salesperson_id, v_period, 'draft', 0, 0,
    'Auto-composed ' || p_period_start::text || ' to ' || p_period_end::text)
  RETURNING id INTO v_payout_id;

  -- Link eligible ledger entries
  UPDATE commission_ledger SET
    payout_id = v_payout_id,
    bonus_pct = v_bonus_pct,
    bonus_amount = commission_amount * v_bonus_pct,
    total_amount = commission_amount * (1 + v_bonus_pct),
    updated_at = now()
  WHERE salesperson_id = p_salesperson_id
    AND status = 'earned'
    AND payout_id IS NULL
    AND earned_at BETWEEN p_period_start AND p_period_end
    AND (p_include_ledger_ids IS NULL OR id = ANY(p_include_ledger_ids));

  -- Update totals on payout row
  SELECT COUNT(*), COALESCE(SUM(commission_amount), 0), COALESCE(SUM(bonus_amount),
0), COALESCE(SUM(total_amount), 0)
  INTO v_count, v_base_total, v_bonus_applied, v_total
  FROM commission_ledger WHERE payout_id = v_payout_id;
```

```
  UPDATE commission_payouts SET
    total_amount = v_total,
    commission_count = v_count
  WHERE id = v_payout_id;

  RETURN jsonb_build_object(
    'payout_id', v_payout_id,
    'count', v_count,
    'base_total', v_base_total,
    'bonus_applied', v_bonus_applied,
    'bonus_pct', v_bonus_pct,
    'goal_attainment', v_goal_attainment,
    'total', v_total
  );
END;
$$;
```

6.3 RPC: approve_commission_payout

Transition draft → approved. Locks the ledger entries. Inserts immutable snapshot into commission_payout_snapshots.

```
CREATE OR REPLACE FUNCTION approve_commission_payout(
  p_payout_id uuid,
  p_approved_by uuid DEFAULT NULL  -- defaults to auth.uid()
) RETURNS commission_payouts
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE v_payout commission_payouts%ROWTYPE;
BEGIN
  SELECT * INTO v_payout FROM commission_payouts WHERE id = p_payout_id FOR UPDATE;
  IF v_payout.status != 'draft' THEN
    RAISE EXCEPTION 'Payout is not in draft status';
  END IF;

  -- Snapshot
  INSERT INTO commission_payout_snapshots (payout_id, snapshot)
  SELECT p_payout_id, jsonb_build_object(
    'payout', row_to_json(cp),
    'ledger', (SELECT jsonb_agg(row_to_json(cl)) FROM commission_ledger cl WHERE
cl.payout_id = p_payout_id),
    'rep_rates', (SELECT jsonb_agg(row_to_json(cr)) FROM commission_rates cr WHERE
cr.salesperson_id = cp.salesperson_id),
    'goals', (SELECT jsonb_agg(row_to_json(cg)) FROM commission_issue_goals cg
              WHERE cg.issue_id IN (SELECT DISTINCT issue_id FROM commission_ledger
WHERE payout_id = p_payout_id))
```

```
  )
  FROM commission_payouts cp WHERE cp.id = p_payout_id;

  UPDATE commission_payouts SET
    status = 'approved',
    approved_by = COALESCE(p_approved_by, auth.uid()),
    approved_at = now()
  WHERE id = p_payout_id
  RETURNING * INTO v_payout;

  RETURN v_payout;
END;
$$;
```

6.4 RPC: mark_commission_payout_paid

Transition approved → paid. Stamps commission_ledger.status = paid, paid_at. Inserts activity_log entry per rep. Optionally queues QuickBooks payroll sync job (future).

```
CREATE OR REPLACE FUNCTION mark_commission_payout_paid(
  p_payout_id uuid,
  p_payment_method text DEFAULT 'check',
  p_reference_number text DEFAULT NULL,
  p_notes text DEFAULT NULL
) RETURNS commission_payouts
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE v_payout commission_payouts%ROWTYPE;
BEGIN
  SELECT * INTO v_payout FROM commission_payouts WHERE id = p_payout_id FOR UPDATE;
  IF v_payout.status != 'approved' THEN
    RAISE EXCEPTION 'Payout must be approved before marking paid';
  END IF;

  UPDATE commission_ledger SET
    status = 'paid',
    paid_at = now()
  WHERE payout_id = p_payout_id;

  UPDATE commission_payouts SET
    status = 'paid',
    paid_at = now(),
    notes = COALESCE(notes, '') || E'\n' || COALESCE(p_notes, 'Paid via ' ||
p_payment_method)
  WHERE id = p_payout_id
  RETURNING * INTO v_payout;
```

```
  INSERT INTO activity_log (type, actor_id, detail)
  VALUES ('commission_paid', auth.uid(),
    'Payout ' || p_payout_id || ': $' || v_payout.total_amount || ' to ' ||
    (SELECT name FROM team_members WHERE id = v_payout.salesperson_id));

  RETURN v_payout;
END;
$$;
```

6.5 RPC: void_commission_payout

Reverses approval. Releases ledger entries back to earned (clears payout_id). Flips payout status to void with reason. Used rarely — incorrect period, accidentally-approved duplicate.

```
CREATE OR REPLACE FUNCTION void_commission_payout(
  p_payout_id uuid,
  p_reason text
) RETURNS commission_payouts
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE v_payout commission_payouts%ROWTYPE;
BEGIN
  SELECT * INTO v_payout FROM commission_payouts WHERE id = p_payout_id FOR UPDATE;
  IF v_payout.status = 'paid' THEN
    RAISE EXCEPTION 'Cannot void a paid payout; issue a reversing adjustment
instead';
  END IF;

  UPDATE commission_ledger SET
    status = 'earned',
    payout_id = NULL,
    bonus_pct = 0,
    bonus_amount = 0,
    total_amount = commission_amount
  WHERE payout_id = p_payout_id;

  UPDATE commission_payouts SET
    status = 'void',
    notes = COALESCE(notes, '') || E'\nVoided: ' || p_reason
  WHERE id = p_payout_id
  RETURNING * INTO v_payout;

  RETURN v_payout;
END;
$$;
```

6.6 New table: commission_payout_snapshots

Immutable audit record for every approved payout. Stores a JSON snapshot of the payout, linked ledger entries, rep's rates, and issue goals at the time of approval. This is the defensible record if a rep ever disputes a payout.

Column	Type	Purpose
`id`	`uuid PK`	Primary key.
`payout_id`	`uuid FK → commission_payouts`	ON DELETE RESTRICT. No cascade — snapshot outlives payout row.
`snapshot`	`jsonb`	Full state: payout row, ledger rows, rates, goals.
`snapshot_sha`	`text`	SHA-256 of snapshot for tamper detection.
`created_at`	`timestamptz`	Default now().

6.7 UI requirements

6.7.1 Commission Run Composer (publisher)

Three-step modal flow, publisher-only:

- **Step 1 — Period selection.** Rep picker (one or batch all), period type (monthly / bi-weekly / custom). Preview count of eligible entries + total. Does NOT create anything yet.
- **Step 2 — Review draft.** Table of linked ledger entries grouped by publication + issue. Bonus tier shown with goal attainment percentage. Publisher can deselect entries (reserves them for next period). "Approve" button finalizes — snapshot captured, ledger locked.
- **Step 3 — Mark paid.** Enter payment method, reference number (check #, ACH id), notes. "Mark paid" finalizes.

6.7.2 Rep self-service Commission tab

Each rep's dashboard has a Commission tab showing:

- YTD earned + YTD paid summary cards.
- Current period progress: earned-so-far vs goal, with bonus tier indicator (tier lights up as the rep crosses thresholds).
- Next payout date (from team_members.commission_payout_frequency).
- Ledger table with filters: period, publication, status. Each row shows sale → commission calculation → status.
- Past payouts table with downloadable statement PDF (generated from snapshot).

6.7.3 Goal attainment visualization

The bonus tier mechanic only makes sense if the rep can see where they are against goal. Each issue goal becomes a visible progress bar on the rep's commission tab during the period, updating in real time as closed sales come in.

6.8 Migration 122 — commission payout execution

```
-- 122_commission_payout_execution.sql

-- 1. Snapshots table
CREATE TABLE commission_payout_snapshots (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  payout_id uuid NOT NULL REFERENCES commission_payouts(id) ON DELETE RESTRICT,
  snapshot jsonb NOT NULL,
  snapshot_sha text GENERATED ALWAYS AS (encode(sha256(snapshot::text::bytea),
'hex')) STORED,
  created_at timestamptz DEFAULT now()
);
CREATE INDEX cps_payout ON commission_payout_snapshots(payout_id);

-- 2. Extend commission_payouts status enum constraint
ALTER TABLE commission_payouts DROP CONSTRAINT IF EXISTS
commission_payouts_status_check;
ALTER TABLE commission_payouts ADD CONSTRAINT commission_payouts_status_check
  CHECK (status IN ('draft','approved','paid','void'));

-- 3. RPCs from Sections 6.2 - 6.5

-- 4. Share pct helper (used by compose_commission_payout)
-- If commission_issue_goals.share_pct does not already exist, add it
ALTER TABLE commission_issue_goals
  ADD COLUMN IF NOT EXISTS share_pct numeric(5,2) DEFAULT 100.00;

-- 5. RLS: publisher role only for INSERT/UPDATE on commission_payouts
-- Reps can SELECT their own
ALTER TABLE commission_payouts ENABLE ROW LEVEL SECURITY;
CREATE POLICY commission_payouts_select ON commission_payouts
  FOR SELECT USING (
    salesperson_id = (SELECT id FROM team_members WHERE auth_id = auth.uid())
    OR EXISTS (SELECT 1 FROM team_members tm WHERE tm.auth_id = auth.uid()
               AND tm.global_role IN ('publisher', 'admin', 'editor_in_chief'))
  );
CREATE POLICY commission_payouts_write ON commission_payouts
  FOR ALL USING (
    EXISTS (SELECT 1 FROM team_members tm WHERE tm.auth_id = auth.uid()
            AND tm.global_role IN ('publisher', 'admin'))
  );
```

6.9 Acceptance criteria

- Hayley runs the first real monthly payout in under 5 minutes per rep.
- After approval, the ledger entries for that rep cannot be recomputed or re-paid accidentally.
- Dana can log in and see exactly which sales contributed to her current and past commissions, grouped by publication and issue.
- A voided payout returns the ledger entries to earned status so they can be included in a corrected run.
- Snapshot SHA matches after 6 months (tamper detection: any change to the ledger row would change the JSON representation and break the hash).
- Goal attainment tier is correctly computed when a rep crosses 100%, 110%, 120%, 130%, 140%+ against weighted goals.

7. Gap 5 — Sales Rep Dashboard

This is the anchor view. It is what Dana and Christie open at 8am and close at 5pm, and what their entire day routes through. Every other gap in this spec feeds it. Build this last — after Gaps 1 through 4 are applied — because it has no entities of its own; it is entirely a surfacing problem.

Design principle

The dashboard answers four questions at a glance: What did I win? What am I working? What's slipping away? What's at risk of going away?

Nothing is more than two clicks from action. A rep sees a Renewal card, clicks Draft, reviews the proposal, clicks Send — that is three clicks.

Every number traces to a source. No summaries without drill-down. If a card shows "$18,400 pipeline," the rep can click it to see the five opportunities that add up to $18,400.

7.1 Information architecture

One dashboard URL: /dashboard. Rep lens by default. Publisher sees the same dashboard with a rep-picker dropdown and an "All Reps" aggregated option.

Six zones, always visible, reorderable via preference:

Revenue Command Bar (top)	KPIs only: MTD closed, pipeline weighted, YTD to goal, next payout amount. Always visible, horizontal strip.
Renewal Queue (left column, prime)	Urgent / Renewal / Lapsed tabs. The action zone.
Pipeline Kanban (center, prime)	Opportunities by stage. Drag-drop advances stages. Cards show client name, value, staleness.
Today's Actions (right column)	Items due today: follow-ups, proofs waiting, reminders. Unified agenda.
Unmatched Comms (banner, conditional)	Appears only when Gmail ingest has unresolved comms. One-click assign.
Recent Activity (bottom)	Chronological: what changed in my clients in the last 24 hours.

7.2 Revenue Command Bar

Four KPI cards across the top, each with a primary number and a secondary context metric. Numbers recompute on every page load; no caching.

7.2.1 Cards

Card 1 — MTD Closed

- Primary: dollar value of sales with date in current month and status = Closed, where sale.assigned_to = rep (or in rep's pub_shares at share %).

- Secondary: day-of-month progress vs monthly goal. "Day 14 of 30 · 62% of target."
- Color: green if on pace, amber if within 90%, red if behind.
- Click → drill into the MTD sales list.

Card 2 — Weighted Pipeline

- Primary: SUM(estimated_value * stage_weight) across all open opportunities where assigned_to = rep.
- Secondary: count of opportunities + average days-in-stage.
- Click → Pipeline Kanban focus-filtered to this rep.

Card 3 — YTD to Goal

- Primary: YTD closed dollars / YTD goal. "78% of year."
- Secondary: forecast end-of-year based on pipeline + historical close rate.
- Click → annual performance view.

Card 4 — Next Payout

- Primary: amount earned in current unpaid ledger entries.
- Secondary: bonus tier status ("110% of goal — 4% bonus unlocked"), next payout date.
- Click → Commission tab.

7.3 Renewal Queue (Gap 3 UI — lives here)

Three-tab strip inside a single card: Urgent (red), Renewal (amber), Lapsed (gray). Count badge on each. Spec details in Section 5.6.1.

Zone-specific behavior on the dashboard:

- Default tab on load = whichever has the highest count among Urgent and Renewal.
- Zero state: "No renewals in your window" — not a blank card. Show the next upcoming (61+ days) as a secondary note.
- Each card is draggable onto the Pipeline Kanban to explicitly convert to an opportunity (if not auto-created).

7.4 Pipeline Kanban

The Phase 1 view from Gap 1, adapted for the dashboard context. Full detail in Section 3.6.1.

Dashboard-specific behavior:

- Columns: Discovery, Qualifying, Proposal, Negotiation (4 columns, not 6 — Won/Lost are collapsed to small count badges at top right).
- Max 10 cards per column visible at once; "show all (X more)" link expands.
- Default sort per column: by staleness (oldest last-interaction first — "nudge me toward what I'm ignoring").
- Inline quick-actions on card: "+1 call," "+1 email," "Set next action" without opening the full detail view.

7.5 Today's Actions

Unified agenda. Everything the rep needs to touch today, collapsed to a single list sorted by priority then time.

Sources combined into this list:

- **Follow-ups due today.** opportunities WHERE next_action_date = today AND assigned_to = rep.
- **Stale opportunities.** opportunities WHERE last_interaction_at < now() - 14 days AND stage NOT IN (Won, Lost). Surfaced as "Hasn't heard from you" nudges.
- **Proposals awaiting response.** proposals WHERE status = Sent AND (now() - sent_at) > 5 days AND assigned_to = rep.
- **Proofs pending client approval.** ad_projects WHERE status IN ('proof_sent') AND client's rep = me, older than 3 days.
- **Billing exceptions.** invoices for rep's clients in suppressed_for_credit_hold OR auto_charge_attempts >= 3.
- **Failed Gmail matches.** interactions WHERE opportunity_id IS NULL AND client_id IS NULL AND the match heuristic points to this rep's known contacts.

Card format per item:

- Icon (type), client name, short description, action button (type-specific: "Log call", "Send", "Reassign", etc.).
- Swipe-left to dismiss (doesn't close the task, just hides from today's list).
- Swipe-right to complete (for items that have a clear completion state).

7.6 Unmatched Comms banner

Appears at the top of the dashboard only when the rep has unassigned Gmail ingest interactions. Spec details in Section 3.6.3.

7.7 Recent Activity feed

Chronological feed of everything that happened in the last 24 hours across the rep's book. Read-only; all items link to their source entity.

Source queries:

- activity_log entries for this rep's clients in the last 24 hours.
- interactions created in the last 24 hours for this rep's clients (even auto-ingested).
- proposal_signatures viewed_at or signed_at in the last 24 hours for this rep's proposals.
- payments received on this rep's clients' invoices.

Combined, grouped by client (so an advertiser's events cluster), most recent first. Cap at 50 items per view with pagination.

7.8 Views and queries

A set of SQL views power the dashboard cheaply. All read-only; all filtered by the authed rep's team_members.id or by an explicit rep_id parameter (publisher view).

v_rep_revenue_kpis

```
CREATE OR REPLACE VIEW v_rep_revenue_kpis AS
SELECT
  tm.id AS rep_id,
  tm.name AS rep_name,
  -- MTD closed
  COALESCE(SUM(s.amount) FILTER (
    WHERE s.status = 'Closed'
      AND s.closed_at >= DATE_TRUNC('month', CURRENT_DATE)
      AND s.closed_at < DATE_TRUNC('month', CURRENT_DATE) + interval '1 month'
  ), 0) AS mtd_closed,
  -- YTD closed
  COALESCE(SUM(s.amount) FILTER (
    WHERE s.status = 'Closed'
      AND s.closed_at >= DATE_TRUNC('year', CURRENT_DATE)
  ), 0) AS ytd_closed,
  -- Weighted pipeline (opportunities)
  COALESCE((
    SELECT SUM(estimated_value * stage_weight(stage))
    FROM opportunities
    WHERE assigned_to = tm.id AND stage NOT IN ('Won','Lost')
  ), 0) AS weighted_pipeline,
  -- Next payout
  COALESCE((
    SELECT SUM(total_amount) FROM commission_ledger
    WHERE salesperson_id = tm.id AND status = 'earned' AND payout_id IS NULL
  ), 0) AS unpaid_earned
```

```
FROM team_members tm
LEFT JOIN sales s ON s.assigned_to = tm.id
WHERE tm.role IN ('Salesperson','Sales Manager')
  AND tm.is_active = true
GROUP BY tm.id, tm.name;
```

v_rep_today_actions

Materializes the "Today's Actions" list into one query. Each row is one action item.

```
CREATE OR REPLACE VIEW v_rep_today_actions AS
-- Union of 6 sub-queries, one per source type (follow-ups, stale, proofs, etc.)
-- Each returns: rep_id, action_type, entity_type, entity_id, client_id,
client_name,
-- priority, description, due_at, metadata jsonb
-- Full implementation in migration 123.
```

v_rep_pipeline_kanban

Stage-grouped opportunity snapshot with join to latest interaction.

7.9 Layout spec

Desktop-first. Breakpoints at 1024 and 1280. On mobile, zones stack in priority order (Revenue Bar → Today's Actions → Renewals → Pipeline → Activity).

Desktop layout (1280+)

```
┌──────────────────────────────────────────────────────────────────┐
│                    REVENUE COMMAND BAR                           │
│  [MTD Closed]   [Pipeline]   [YTD Goal]   [Next Payout]          │
├──────────────────────────────────────────────────────────────────┤
│                      [Unmatched Comms banner]                    │
├────────────────────┬──────────────────────────┬──────────────────┤
│                    │                          │                  │
│  RENEWAL QUEUE     │   PIPELINE KANBAN        │  TODAY'S ACTIONS │
│  [Urgent]          │                          │                  │
│  [Renewal]         │   Disc | Qual |          │   Follow-ups     │
│  [Lapsed]          │   Prop | Neg             │   Stale opps     │
│                    │                          │   Proofs         │
│  Client cards      │   Draggable cards        │   Billing alerts │
│                    │                          │                  │
│  width: 320px      │   flex: 1                │   width: 360px   │
├────────────────────┴──────────────────────────┴──────────────────┤
│                                                                  │
│                    RECENT ACTIVITY FEED                          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Responsive (1024-1280)

Renewal Queue collapses to tabs inside the Pipeline Kanban column, freeing horizontal space. Today's Actions stays right-column.

Mobile (<1024)

Single-column stack. Navigation: bottom tab bar with five tabs matching the zones. Revenue Command Bar always at top.

7.10 Publisher view

Same layout, with a rep picker at the top and an "All Reps" aggregated option. Publisher can also see the dashboard as any rep (impersonation) for troubleshooting.

All Reps aggregation changes:

- Revenue Command Bar shows totals across reps plus a per-rep breakdown mini-chart.
- Renewal Queue: all renewals for all reps, with assigned-to avatar on each card.
- Pipeline Kanban: same columns, cards show rep avatar; optional "group by rep" toggle.
- Today's Actions: across-team agenda sorted by rep, then time.

7.11 Migration 123 — dashboard views + RPCs

```
-- 123_sales_rep_dashboard.sql

-- 1. v_rep_revenue_kpis (see 7.8)
-- 2. v_rep_today_actions (union of 6 source queries)
-- 3. v_rep_pipeline_kanban
-- 4. v_rep_recent_activity

-- RPC: get_rep_dashboard — single call that returns all dashboard data
CREATE OR REPLACE FUNCTION get_rep_dashboard(p_rep_id uuid DEFAULT NULL)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_rep uuid := COALESCE(p_rep_id, (SELECT id FROM team_members WHERE auth_id =
auth.uid()));
  v_result jsonb;
BEGIN
  SELECT jsonb_build_object(
    'kpis', (SELECT row_to_json(k) FROM v_rep_revenue_kpis k WHERE k.rep_id =
v_rep),
```

```
    'renewal_queue', (SELECT jsonb_agg(row_to_json(r)) FROM v_renewal_queue r WHERE
r.rep_id = v_rep),
    'pipeline', (SELECT jsonb_agg(row_to_json(p)) FROM v_rep_pipeline_kanban p WHERE
p.rep_id = v_rep),
    'today_actions', (SELECT jsonb_agg(row_to_json(a)) FROM v_rep_today_actions a
WHERE a.rep_id = v_rep),
    'unmatched_comms_count', (SELECT COUNT(*) FROM interactions WHERE opportunity_id
IS NULL AND client_id IS NULL),
    'activity', (SELECT jsonb_agg(row_to_json(ra)) FROM v_rep_recent_activity ra
WHERE ra.rep_id = v_rep LIMIT 50)
  ) INTO v_result;
  RETURN v_result;
END;
$$;

-- A single round-trip to Supabase populates the entire dashboard.
```

7.12 Performance targets

- Full dashboard loads in under 800ms on a rep with 150 open opportunities.
- get_rep_dashboard RPC returns in under 300ms at p95.
- Views use indexed columns only; EXPLAIN ANALYZE shows no sequential scans on any contributing query.
- Client-side: skeleton loaders on each zone; zones render as their data arrives rather than waiting for the slowest.

7.13 Acceptance criteria

- Dana logs in and within one screen sees: what she closed this month, what she's working, what's renewing, and what she needs to do today.
- The four Revenue Command Bar numbers reconcile to the underlying tables within $0 (not rounding error).
- Dragging an opportunity card from Qualifying to Proposal calls advance_opportunity_stage and updates the board in under 200ms optimistically.
- Today's Actions never shows an item that's already been completed (data freshness).
- Publisher viewing "All Reps" sees correct aggregation: total MTD across reps = sum of per-rep MTDs.
- Hayley can impersonate Dana and see exactly what Dana sees, with no additional permissions flipped.

8. Gap 6 — Integration Wiring

The four external services that MyDash depends on for money flow and communication. Each has partial schema already in place (columns on invoices, payments, clients, team_members); none is live end-to-end. This section specs the worker Edge Functions, OAuth token storage, retry behavior, and dead-letter handling that turn the queue from Section 4 into a reliable integration backbone.

Architectural choice: pull-based worker, not push

Every integration is driven by the integration_jobs queue from Section 4. Workers poll on a cron (every minute); no webhooks into Supabase directly. Reasons: simpler auth, easier to reason about retries, every job is audit-logged before it runs, and Supabase Edge Function cold starts are cheap on the Small compute tier.

Exceptions: Stripe webhooks (charge.succeeded, charge.failed) fire into a single receiver Edge Function that enqueues follow-up jobs. Gmail Push notifications also enqueue. Both sources are push into the queue, not push to act.

8.1 OAuth token storage

All four integrations require OAuth. Tokens are stored in a single integration_credentials table, scoped by team member + provider. Publisher-level service accounts are stored with team_id = null and a sentinel provider_scope = org.

Column	Type	Purpose
`id`	`uuid PK`	Primary key.
`team_member_id`	`uuid FK → team_members`	NULL for org-level credentials.
`provider`	`integration_provider enum`	quickbooks, stripe, gmail, google_calendar. NOT NULL.
`provider_scope`	`text`	rep (per-user tokens) or org (shared). NOT NULL.
`access_token`	`text`	Encrypted at rest via pgsodium. NOT NULL.
`refresh_token`	`text`	Encrypted. Nullable (Stripe uses API keys, not refresh flow).
`expires_at`	`timestamptz`	When access_token expires. Workers refresh 5 min before.
`scopes`	`text[]`	Granted scopes, for audit.
`external_account_id`	`text`	QBO realmId, Stripe account ID, Google email. NOT NULL.
`connected_at`	`timestamptz`	Default now().
`last_refreshed_at`	`timestamptz`	
`last_used_at`	`timestamptz`	Updated by every worker call; drives "still connected" UI.
`status`	`credential_sta`	active, expired, revoked, error. Default active.

Column	Type	Purpose
	`tus enum`	
`error_message`	`text`	Last error if status is error.

Token refresh happens at the worker level — before any external call, the worker checks expires_at and refreshes if needed. Refresh is atomic (row lock) to prevent multiple workers refreshing concurrently.

8.2 QuickBooks Online

QBO is the bookkeeping source of truth. Every invoice and payment in MyDash syncs to QBO; nothing syncs FROM QBO back (MyDash is authoritative for the sales cycle). The integration is one-way write.

8.2.1 Sync scope

Component	Status	Notes
Clients → QBO Customers	**WIRED (create-only)**	On first invoice sync; stripe_customer_id stored; no update propagation
Invoices → QBO Invoices	**NEEDS WORKER**	Fired by press cascade (Section 4); includes lines + tax + terms
Payments → QBO Payments	**NEEDS WORKER**	Fired when payments row inserted; applies to QBO invoice by quickbooks_id
Credit memos	**FUTURE**	Out of scope for initial build
Invoice voids	**NEEDS WORKER**	Separate job type; fires on invoice.status → void

8.2.2 Worker: qbo-invoice-sync

Edge Function at supabase/functions/qbo-invoice-sync. Cron: * * * * * (every minute). Claims up to 20 queued jobs per invocation.

```
// supabase/functions/qbo-invoice-sync/index.ts
import { createClient } from '@supabase/supabase-js';

const WORKER_ID = `qbo-inv-${crypto.randomUUID()}`;

Deno.serve(async () => {
  const supabase = createClient(Deno.env.get('SUPABASE_URL')!,
Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!);
  const results = [];

  // 1. Claim up to 20 jobs atomically
  const { data: jobs } = await supabase.rpc('claim_integration_jobs', {
```

```
    p_worker_id: WORKER_ID,
    p_types: ['qbo_invoice_create', 'qbo_invoice_update', 'qbo_invoice_void'],
    p_limit: 20,
    p_lock_minutes: 5
  });

  // 2. Fetch org-level QBO credentials
  const { data: creds } = await supabase.from('integration_credentials')
    .select('*').eq('provider', 'quickbooks').eq('provider_scope', 'org').single();

  // 3. Refresh token if expiring
  const freshCreds = await ensureFresh(supabase, creds);

  // 4. Process each job
  for (const job of jobs ?? []) {
    try {
      await processQboJob(supabase, freshCreds, job);
      await supabase.from('integration_jobs').update({
        status: 'succeeded', result: { processed_at: new Date().toISOString() },
        locked_by: null, locked_until: null
      }).eq('id', job.id);
      results.push({ id: job.id, status: 'ok' });
    } catch (err) {
      await handleJobFailure(supabase, job, err);
      results.push({ id: job.id, status: 'fail', error: String(err) });
    }
  }

  return new Response(JSON.stringify({ claimed: jobs?.length ?? 0, results }));
});

async function processQboJob(supabase, creds, job) {
  const { invoice_id } = job.payload;
  const { data: inv } = await supabase.from('invoices').select(
    '*, client:clients(*), lines:invoice_lines(*, publication:publications(*))'
  ).eq('id', invoice_id).single();

  // Ensure customer exists in QBO
  let customerId = inv.client.quickbooks_customer_id;
  if (!customerId) {
    customerId = await qboCreateCustomer(creds, inv.client);
    await supabase.from('clients').update({ quickbooks_customer_id: customerId })
      .eq('id', inv.client.id);
  }

  if (job.type === 'qbo_invoice_create') {
    const qboId = await qboCreateInvoice(creds, inv, customerId);
    await supabase.from('invoices').update({
      quickbooks_id: qboId,
```

```
      quickbooks_synced_at: new Date().toISOString(),
      quickbooks_sync_error: null
    }).eq('id', invoice_id);
  } else if (job.type === 'qbo_invoice_update') {
    await qboUpdateInvoice(creds, inv);
  } else if (job.type === 'qbo_invoice_void') {
    await qboVoidInvoice(creds, inv.quickbooks_id);
  }
}
```

8.2.3 QBO line mapping

MyDash invoice lines map to QBO line items with these conventions:

MyDash invoice_line.description	QBO SalesItemLineDetail.Description
MyDash invoice_line.amount	QBO SalesItemLineDetail.LineNum amount
MyDash publication name	QBO Class (for per-pub P&L reporting)
MyDash issue.label	QBO Line.Description prefix ("PRP 2026-04-15: ...")
MyDash product_type	QBO Item (pre-configured items per product_type in QBO)
MyDash invoice.due_date	QBO DueDate
MyDash invoice.invoice_number	QBO DocNumber

8.2.4 Worker: qbo-payment-sync

Same pattern as invoice sync. Fires when payments row is inserted with quickbooks_id = null. Posts to QBO Payment endpoint, applies to one or more QBO invoices by quickbooks_id.

```
// supabase/functions/qbo-payment-sync/index.ts
// Triggered by: integration_jobs row with type = qbo_payment_create
// Enqueued by: trigger on payments table AFTER INSERT

CREATE OR REPLACE FUNCTION tf_enqueue_qbo_payment_sync()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO integration_jobs (type, payload, related_entity_type,
related_entity_id)
  VALUES ('qbo_payment_create', jsonb_build_object('payment_id', NEW.id),
          'payment', NEW.id::text);
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS tr_enqueue_qbo_payment_sync ON payments;
CREATE TRIGGER tr_enqueue_qbo_payment_sync
  AFTER INSERT ON payments
```

```
  FOR EACH ROW
  EXECUTE FUNCTION tf_enqueue_qbo_payment_sync();
```

8.3 Stripe

Stripe handles automatic card charging when an invoice is sent on press. Two worker functions plus a webhook receiver.

8.3.1 Worker: stripe-auto-charge

Cron every minute. Claims queued stripe_auto_charge jobs. Each job charges one invoice using the client's saved payment method.

```
// supabase/functions/stripe-auto-charge/index.ts
import Stripe from 'https://esm.sh/stripe@14?target=deno';

const stripe = new Stripe(Deno.env.get('STRIPE_SECRET_KEY')!, { apiVersion: '2024-
04-10' });

async function chargeInvoice(supabase, job) {
  const { invoice_id } = job.payload;
  const { data: inv } = await supabase.from('invoices').select(
    '*, client:clients(stripe_customer_id, stripe_payment_method_id, credit_hold)'
  ).eq('id', invoice_id).single();

  if (inv.status === 'paid') { return { skipped: 'already_paid' }; }
  if (inv.client.credit_hold) { return { skipped: 'credit_hold' }; }
  if (!inv.client.stripe_payment_method_id) { return { skipped: 'no_card' }; }

  // Idempotency key prevents double-charge on worker retry
  const idempotencyKey = `inv_${invoice_id}_attempt_${job.attempts + 1}`;

  try {
    const intent = await stripe.paymentIntents.create({
      amount: Math.round(inv.amount_due * 100),  // cents
      currency: 'usd',
      customer: inv.client.stripe_customer_id,
      payment_method: inv.client.stripe_payment_method_id,
      off_session: true,
      confirm: true,
      description: `Invoice ${inv.invoice_number}`,
      metadata: { mydash_invoice_id: invoice_id, mydash_client_id: inv.client_id }
    }, { idempotencyKey });

    await supabase.from('invoices').update({
      stripe_payment_intent_id: intent.id,
      auto_charge_attempts: (inv.auto_charge_attempts ?? 0) + 1,
      last_charge_attempt: new Date().toISOString(),
```

```
      charge_error: null
    }).eq('id', invoice_id);

    // Payment row + invoice.status flip happens via webhook, not here.
    // This keeps charge.succeeded as the single source of truth.
    return { intent_id: intent.id, status: intent.status };
  } catch (err) {
    await supabase.from('invoices').update({
      auto_charge_attempts: (inv.auto_charge_attempts ?? 0) + 1,
      last_charge_attempt: new Date().toISOString(),
      charge_error: err.message
    }).eq('id', invoice_id);
    throw err;
  }
}
```

8.3.2 Retry strategy

Card decline is not always terminal — often a temporary hold, insufficient funds, or expired card. Retry schedule:

Attempt 1	Immediate (on press cascade) — after 5-minute buffer
Attempt 2	Attempt 1 + 24 hours
Attempt 3	Attempt 2 + 48 hours
Attempt 4	Attempt 3 + 7 days
Attempt 5 → Dead letter	Attempt 4 + 7 days · invoice continues normal reminder cadence

After attempt 5 fails, the job is marked dead. The invoice reverts to the standard manual-pay track — reminders continue as scheduled, rep gets a billing alert on the dashboard. Office Admin can manually retry by clicking a "Retry Auto-Charge" button which resets attempts to 0 and re-queues the job.

8.3.3 Webhook receiver: stripe-webhook

Single Edge Function that verifies Stripe signature and enqueues follow-up jobs. Never writes to MyDash tables directly — delegates to workers.

```
// supabase/functions/stripe-webhook/index.ts
Deno.serve(async (req) => {
  const sig = req.headers.get('stripe-signature');
  const body = await req.text();
  const event = stripe.webhooks.constructEvent(body, sig,
Deno.env.get('STRIPE_WEBHOOK_SECRET')!);

  const supabase = createClient(...);
```

```
  if (event.type === 'payment_intent.succeeded') {
    await supabase.from('integration_jobs').insert({
      type: 'stripe_payment_succeeded',
      payload: {
        payment_intent_id: event.data.object.id,
        amount: event.data.object.amount / 100,
        mydash_invoice_id: event.data.object.metadata?.mydash_invoice_id
      },
      related_entity_type: 'invoice',
      related_entity_id: event.data.object.metadata?.mydash_invoice_id,
      priority: 1  // high priority
    });
  } else if (event.type === 'payment_intent.payment_failed') {
    // Similar enqueue for failure handling
  } else if (event.type === 'payment_method.attached') {
    // Auto-update client.stripe_payment_method_id
  }

  return new Response(JSON.stringify({ received: true }));
});

// Webhook URL: https://<project>.supabase.co/functions/v1/stripe-webhook
// Must be configured in Stripe dashboard with HMAC signing secret in env.
```

8.3.4 Worker: stripe-payment-succeeded-handler

Consumes stripe_payment_succeeded jobs (enqueued by webhook). Creates payments row and calls apply_payment_to_invoices, which cascades through the existing waterfall logic.

8.4 Gmail

Gmail integration has two purposes: ingest inbound and outbound messages into the interactions table (Section 3.3), and send outbound emails (reminders, proposals, acknowledgments). Both use the same OAuth credentials but different scopes.

8.4.1 Setup: Gmail Push notifications

Rather than polling every rep's inbox, Gmail Pub/Sub fires a notification to a webhook when new messages arrive. Setup steps:

- **1. Google Cloud Pub/Sub topic.** Created once per organization: projects/13stars-mydash/topics/gmail-ingest.
- **2. Watch request per rep.** On rep OAuth completion, call users.watch() with the topic and INBOX label filter. Returns historyId for subsequent polling.
- **3. Webhook receives notifications.** Google sends JSON {"emailAddress": "...", "historyId": "..."} to our webhook. We enqueue a gmail_ingest_poll job with that historyId.

- **4. Weekly watch refresh.** watch() expires after 7 days; a cron job re-registers every 6 days.

8.4.2 Worker: gmail-ingest

```
// supabase/functions/gmail-ingest/index.ts
// Processes gmail_ingest_poll jobs from the queue.

async function processGmailPoll(supabase, job) {
  const { email_address, history_id } = job.payload;

  // Find rep by email
  const { data: rep } = await supabase.from('team_members')
    .select('id, gmail_ingest_enabled').eq('email', email_address).single();
  if (!rep?.gmail_ingest_enabled) return { skipped: 'ingest_disabled' };

  // Get rep's Gmail credentials
  const creds = await getCredentials(supabase, rep.id, 'gmail');

  // Fetch history since last processed
  const lastHistory = await getLastHistoryId(supabase, rep.id);
  const history = await gmailHistory(creds, lastHistory ?? history_id);

  for (const msg of history.messages ?? []) {
    const full = await gmailGetMessage(creds, msg.id);
    const parsed = parseMessage(full);  // extract from/to/subject/body, strip
quotes

    if (parsed.bodyDirection === 'internal') continue;  // team internal emails

    await supabase.rpc('ingest_gmail_interaction', {
      p_external_id: msg.id,
      p_thread_external_id: full.threadId,
      p_from_email: parsed.from,
      p_to_emails: parsed.to,
      p_subject: parsed.subject,
      p_body: parsed.body,
      p_occurred_at: parsed.date,
      p_rep_team_id: rep.id,
      p_sentiment: await classifySentiment(parsed.body),  // optional AI step
      p_attachments: parsed.attachments
    });
  }

  // Advance our watermark
  await setLastHistoryId(supabase, rep.id, history.historyId);
}
```

8.4.3 Outbound email send

Proposal send, reminder send, and rep-composed messages all use the same send path. Uses the rep's OAuth token when available (sends appear in their Sent folder), falls back to a noreply@ org-level SMTP account.

```
// supabase/functions/gmail-send/index.ts
async function sendEmail(supabase, job) {
  const { from_team_id, to_emails, subject, html_body, text_body, attachments } =
job.payload;

  let creds = from_team_id
    ? await getCredentials(supabase, from_team_id, 'gmail')
    : await getOrgCredentials(supabase, 'gmail');  // noreply@ fallback

  const mime = buildMimeMessage({ to: to_emails, subject, html_body, text_body,
attachments });
  const encoded = base64UrlEncode(mime);

  const res = await
fetch('https://gmail.googleapis.com/gmail/v1/users/me/messages/send', {
    method: 'POST',
    headers: { Authorization: `Bearer ${creds.access_token}`, 'Content-Type':
'application/json' },
    body: JSON.stringify({ raw: encoded })
  });
  const sent = await res.json();

  // Optionally log as an outbound interaction
  if (job.payload.log_as_interaction) {
    await supabase.from('interactions').insert({
      opportunity_id: job.payload.opportunity_id,
      client_id: job.payload.client_id,
      type: 'email', direction: 'outbound',
      occurred_at: new Date().toISOString(),
      subject, body: text_body,
      external_id: sent.id, thread_external_id: sent.threadId,
      source: 'manual', logged_by: from_team_id
    });
  }
}
```

8.5 Google Calendar

Calendar integration syncs opportunity follow-ups and meetings as events in the rep's calendar. Two-way: create events from MyDash, and ingest events tagged with a client contact back as interactions.

8.5.1 Event creation (MyDash → Calendar)

When a rep sets next_action_date on an opportunity (via the opportunity detail view or a log_interaction call with next_action_set = true), a Calendar event is created.

```
async function createFollowupEvent(supabase, job) {
  const { opportunity_id, action_type, action_date, notes } = job.payload;
  const { data: opp } = await supabase.from('opportunities')
    .select('*, client:clients(name), assignee:team_members(email)')
    .eq('id', opportunity_id).single();

  const creds = await getCredentials(supabase, opp.assigned_to, 'google_calendar');

  const event = await
fetch('https://www.googleapis.com/calendar/v3/calendars/primary/events', {
    method: 'POST',
    headers: { Authorization: `Bearer ${creds.access_token}`, 'Content-Type':
'application/json' },
    body: JSON.stringify({
      summary: `${action_type}: ${opp.client.name}`,
      description: `Opportunity: ${opp.name}\n\n${notes ?? ''}`,
      start: { dateTime: action_date, timeZone: 'America/Los_Angeles' },
      end: { dateTime: addHours(action_date, 1), timeZone: 'America/Los_Angeles' },
      extendedProperties: { private: { mydash_opportunity_id: opportunity_id } }
    })
  });
  const created = await event.json();

  await supabase.from('opportunities').update({
    calendar_event_id: created.id,
    calendar_event_link: created.htmlLink
  }).eq('id', opportunity_id);
}
```

8.5.2 Event ingest (Calendar → interactions)

Low-priority integration. Not blocking initial build. When enabled, a daily cron polls the rep's calendar for events that contain client contact emails as attendees; creates interaction rows with type = meeting, source = calendar_ingest.

8.6 Dead-letter queue UI

Publisher-only admin screen at /admin/integrations/dead-jobs. Table of all integration_jobs with status = dead.

- Filterable by type, date range, related entity.
- Row expansion shows full payload + last error.

- Per-row actions: Retry (resets to queued, attempts = 0), Dismiss (status → void, no further retry), Inspect entity (deep-link to related record).
- Count badge in publisher nav when any dead jobs exist.
- Email alert to Hayley when dead_job_count crosses threshold (default 5).

8.7 Connect/disconnect UI

Per-rep Settings > Integrations screen:

Gmail	Connect button → OAuth consent → watch() registered → status: Connected
Google Calendar	Connect button → OAuth (additional scope) → status: Connected
Gmail ingest toggle	Per-rep on/off; off revokes watch() and disables polling

Publisher settings has an additional Org Integrations screen for QuickBooks and Stripe (single org-level connection). Stripe uses API key input (not OAuth); QBO uses full OAuth with realm selection.

8.8 Migration 124 — integration backbone

```
-- 124_integration_backbone.sql

-- 1. Credentials table
CREATE TYPE integration_provider AS ENUM
('quickbooks','stripe','gmail','google_calendar');
CREATE TYPE credential_status AS ENUM ('active','expired','revoked','error');

CREATE TABLE integration_credentials (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  team_member_id uuid REFERENCES team_members(id) ON DELETE CASCADE,
  provider integration_provider NOT NULL,
  provider_scope text NOT NULL,
  access_token text NOT NULL,
  refresh_token text,
  expires_at timestamptz,
  scopes text[],
  external_account_id text NOT NULL,
  connected_at timestamptz DEFAULT now(),
  last_refreshed_at timestamptz,
  last_used_at timestamptz,
  status credential_status NOT NULL DEFAULT 'active',
  error_message text,
  UNIQUE (team_member_id, provider, provider_scope)
);
```

```
-- Partial unique index for org-level credentials (team_member_id IS NULL)
CREATE UNIQUE INDEX integration_credentials_org_unique
  ON integration_credentials(provider, external_account_id)
  WHERE team_member_id IS NULL;

-- 2. Job claim RPC
CREATE OR REPLACE FUNCTION claim_integration_jobs(
  p_worker_id text,
  p_types integration_job_type[],
  p_limit int DEFAULT 10,
  p_lock_minutes int DEFAULT 5
) RETURNS SETOF integration_jobs
LANGUAGE plpgsql
AS $$
BEGIN
  RETURN QUERY
  UPDATE integration_jobs SET
    status = 'running',
    locked_by = p_worker_id,
    locked_until = now() + (p_lock_minutes || ' minutes')::interval,
    last_attempt_at = now(),
    attempts = attempts + 1
  WHERE id IN (
    SELECT id FROM integration_jobs
    WHERE status = 'queued'
      AND next_attempt_at <= now()
      AND type = ANY(p_types)
    ORDER BY priority, created_at
    LIMIT p_limit
    FOR UPDATE SKIP LOCKED
  )
  RETURNING *;
END;
$$;

-- 3. Failure handler RPC
CREATE OR REPLACE FUNCTION handle_job_failure(
  p_job_id uuid,
  p_error text
) RETURNS void
LANGUAGE plpgsql
AS $$
DECLARE
  v_job integration_jobs%ROWTYPE;
  v_backoff int;
BEGIN
  SELECT * INTO v_job FROM integration_jobs WHERE id = p_job_id;
  v_backoff := LEAST(POWER(2, v_job.attempts)::int, 10080);  -- cap at 7 days
  UPDATE integration_jobs SET
```

```
    status = CASE WHEN attempts >= 5 THEN 'dead'::job_status ELSE
'queued'::job_status END,
    error_message = p_error,
    next_attempt_at = now() + (v_backoff || ' minutes')::interval,
    locked_by = NULL,
    locked_until = NULL
  WHERE id = p_job_id;
END;
$$;

-- 4. Stripe payment succeeded type
ALTER TYPE integration_job_type ADD VALUE IF NOT EXISTS 'stripe_payment_succeeded';
ALTER TYPE integration_job_type ADD VALUE IF NOT EXISTS 'stripe_payment_failed';
ALTER TYPE integration_job_type ADD VALUE IF NOT EXISTS 'gmail_send';
ALTER TYPE integration_job_type ADD VALUE IF NOT EXISTS 'qbo_invoice_void';

-- 5. Payment -> QBO sync trigger
CREATE OR REPLACE FUNCTION tf_enqueue_qbo_payment_sync()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO integration_jobs (type, payload, related_entity_type,
related_entity_id)
  VALUES ('qbo_payment_create', jsonb_build_object('payment_id', NEW.id),
          'payment', NEW.id::text);
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS tr_enqueue_qbo_payment_sync ON payments;
CREATE TRIGGER tr_enqueue_qbo_payment_sync
  AFTER INSERT ON payments
  FOR EACH ROW
  EXECUTE FUNCTION tf_enqueue_qbo_payment_sync();

-- 6. Opportunity calendar event link columns
ALTER TABLE opportunities
  ADD COLUMN IF NOT EXISTS calendar_event_id text,
  ADD COLUMN IF NOT EXISTS calendar_event_link text;

-- 7. Team members Gmail ingest toggle
ALTER TABLE team_members
  ADD COLUMN IF NOT EXISTS gmail_ingest_enabled boolean DEFAULT false;

-- 8. Client QBO link
ALTER TABLE clients
  ADD COLUMN IF NOT EXISTS quickbooks_customer_id text;

-- 9. RLS on credentials (reps see their own; publisher sees all)
ALTER TABLE integration_credentials ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY ic_own ON integration_credentials
  FOR ALL USING (
    team_member_id = (SELECT id FROM team_members WHERE auth_id = auth.uid())
    OR EXISTS (SELECT 1 FROM team_members WHERE auth_id = auth.uid()
               AND global_role IN ('publisher','admin'))
  );
```

8.9 Observability

Every integration call is auditable via the integration_jobs table. For active monitoring, a single SQL view surfaces health metrics:

```
CREATE OR REPLACE VIEW v_integration_health AS
SELECT
  type,
  COUNT(*) FILTER (WHERE status = 'queued') AS queued,
  COUNT(*) FILTER (WHERE status = 'running') AS running,
  COUNT(*) FILTER (WHERE status = 'succeeded' AND created_at > now() - interval '1
hour') AS succeeded_last_hour,
  COUNT(*) FILTER (WHERE status = 'failed') AS failed,
  COUNT(*) FILTER (WHERE status = 'dead') AS dead,
  MAX(created_at) FILTER (WHERE status = 'succeeded') AS last_success,
  MAX(created_at) FILTER (WHERE status = 'dead') AS last_dead
FROM integration_jobs
WHERE created_at > now() - interval '30 days'
GROUP BY type;
```

Exposed in the publisher admin as a small health card per integration type, with green/amber/red indicators based on ratios (e.g., dead > 10% of total → red).

8.10 Secrets management

All secrets stored as Supabase Edge Function secrets (set via supabase secrets set). Never in code, never in environment files committed to git.

STRIPE_SECRET_KEY	Live Stripe key for charge creation
STRIPE_WEBHOOK_SECRET	Signing secret for webhook verification
QBO_CLIENT_ID	QuickBooks OAuth app client ID
QBO_CLIENT_SECRET	QuickBooks OAuth app secret
GOOGLE_OAUTH_CLIENT_ID	Google Cloud OAuth client (Gmail + Calendar)
GOOGLE_OAUTH_CLIENT_SECRET	Google Cloud OAuth secret
GOOGLE_PUBSUB_VERIFY_TOKEN	Verification token for Gmail Push callback

BUNNY_API_KEY	Already set — used by image proxy Edge Function
SUPABASE_SERVICE_ROLE_KEY	Set by Supabase platform; used by workers
INTEGRATION_ENCRYPTION_KEY	pgsodium key for token encryption at rest

8.11 Acceptance criteria

- Marking an issue sent to press results in every invoice for that issue appearing in QuickBooks within 5 minutes, with a valid quickbooks_id stored on the MyDash invoice row.
- A client with a saved card on file receives a Stripe charge within 10 minutes of press; successful charge creates a payments row and flips the invoice to paid (via webhook).
- A failed card charge (insufficient funds) is retried 4 times over 16 days before being marked dead; the invoice proceeds through normal reminder cadence in parallel.
- Dana receives an email from a known client contact; within 10 minutes the email appears as an inbound interaction on the correct opportunity without manual action.
- Setting a follow-up date on an opportunity creates a matching event on Dana's Google Calendar with the client name, opportunity name, and context in the description.
- The dead-letter queue never grows unbounded — every dead job is either retried or dismissed within 7 days, enforced by a publisher alert.
- Disconnecting Gmail in rep settings revokes the watch() registration and sets gmail_ingest_enabled = false; no further polling occurs.

9. Migration Plan

Seven migrations, applied in the order below. Each migration is idempotent where possible (using CREATE IF NOT EXISTS, ADD COLUMN IF NOT EXISTS) and designed to apply without downtime on the production database.

Baseline and environment

Starting migration version: 20260424084852 (118_self_serve_creative_workflow). Latest applied as of spec authoring.

Target Supabase project: hqywacyhpllapdwccmaw.

Apply via supabase/migrations/ files committed to git, or via the Supabase MCP apply_migration tool. Recommended: one migration per commit, so git history mirrors schema history.

Remember the MCP session limit — migrations 123 (views) and 124 (integration backbone) both run several DDL statements; either split them across multiple MCP calls, or apply via CLI to avoid the ~30-40 op timeout.

9.1 Migration summary

#	Name	Creates	Depends on	Safe to rollback?
119	opportunity_interaction_model	opportunities, interactions, 8 enums, sales.opportunity_id, stage_weight()	None	Yes
120	press_to_invoice_trigger	invoice_reminders, integration_jobs, press cascade function + trigger, invoice flags	None	Yes (drop trigger)
121	renewal_automation	org_settings renewal cols, updated recompute_client_status, draft_renewal_proposal, v_renewal_queue, run_nightly_renewal_job	119 (needs opportunities)	Mostly — will leave some opportunities created
122	commission_payout_execution	commission_payout_snapshots, 4 RPCs, status constraint, RLS policies	None	Yes
123	sales_rep_dashboard	4 views, get_rep_dashboard RPC	119 (opportunities in views), 121 (renewal view)	Yes — views only
124	integration_backbone	integration_credentials, claim/failure RPCs, new job type enum values, triggers, client.quickbooks_customer_id	120 (integration_jobs table)	Partial — credentials table needs secure cleanup
125 +	Reserved	Reserved for follow-on work (subscriptions, distribution, agent station)	—	—

9.2 Apply order rationale

The dependency graph forces 119 → 121 and 119 → 123 and 120 → 124. Within that constraint, the recommended order minimizes the "half-broken" window:

- **119 first.** Creates the opportunity backbone. Safe to deploy; no existing code reads these tables yet.
- **120 second.** Press trigger is silent until issues are marked sent to press AND the Edge Function workers are deployed. Apply migration; deploy workers later.
- **121 third.** Renewal automation needs opportunities (from 119). The nightly job is inert until the cron is registered separately.
- **122 fourth.** Independent of the others. Can apply any time after baseline.
- **123 fifth.** Views depend on 119 (opportunities) and 121 (v_renewal_queue). Dashboard UI is not deployed yet; views are queryable for manual verification.
- **124 last.** Needs 120's integration_jobs table. Workers are deployed after migration; until then, jobs accumulate in queued status (harmless).

9.3 Per-migration apply commands

Via Supabase CLI from a local clone:

```
cd /path/to/mydash
git checkout -b spec-055-sales-cycle

# Apply one at a time, test, commit
supabase migration new opportunity_interaction_model
# → paste migration 119 body into the generated file
supabase db push
supabase db diff   # verify what landed
git add supabase/migrations/ && git commit -m "spec 055: migration 119 opportunities
+ interactions"

# Repeat for 120..124
```

Via Supabase MCP in a single session:

```
// Pseudocode — invoke apply_migration once per file
apply_migration("119_opportunity_interaction_model", body_of_119);
// wait for success response
apply_migration("120_press_to_invoice_trigger", body_of_120);
// ... and so on
// Split the session if approaching the 30-40 op ceiling (Section 9.7 notes)
```

9.4 Post-migration verification

After each apply, run these smoke queries. They should all succeed against a fresh migration with no data seeded.

After 119

```
SELECT COUNT(*) FROM opportunities; -- expect 0
SELECT enum_range(NULL::opportunity_stage); -- expect 6 values
SELECT stage_weight('Negotiation'); -- expect 0.75
SELECT column_name FROM information_schema.columns
  WHERE table_name = 'sales' AND column_name = 'opportunity_id'; -- expect 1 row
```

After 120

```
SELECT COUNT(*) FROM invoice_reminders; -- expect 0
SELECT COUNT(*) FROM integration_jobs; -- expect 0
SELECT tgname FROM pg_trigger WHERE tgname = 'tr_press_fires_invoices'; -- expect 1
row
```

After 121

```
SELECT renewal_window_days, urgent_renewal_days FROM org_settings; -- expect 60, 30
-- Sanity check: no clients should currently be in Renewal (pre-job)
SELECT run_nightly_renewal_job();
-- Re-check: Active clients with contract_end_date in next 60 days should now be
Renewal
SELECT status, COUNT(*) FROM clients GROUP BY status;
```

After 122

```
-- Compose a test payout against a known rep for last month
SELECT compose_commission_payout(
  (SELECT id FROM team_members WHERE name = 'Dana McGraw'),
  DATE_TRUNC('month', CURRENT_DATE - interval '1 month')::date,
  (DATE_TRUNC('month', CURRENT_DATE) - interval '1 day')::date
);
-- Inspect the returned payout_id, verify commission_payouts + linked ledger
-- Don't approve or pay unless you're doing the real first run
```

After 123

```
SELECT * FROM v_rep_revenue_kpis LIMIT 5;
SELECT * FROM v_renewal_queue LIMIT 5;
SELECT get_rep_dashboard(
  (SELECT id FROM team_members WHERE name = 'Dana McGraw')
);
```

After 124

```
SELECT enum_range(NULL::integration_job_type); -- expect new values included
SELECT proname FROM pg_proc WHERE proname IN
('claim_integration_jobs','handle_job_failure');
```

```
-- Credentials table exists but empty
SELECT COUNT(*) FROM integration_credentials; -- expect 0
```

9.5 Rollback strategy

Each migration ships with a paired rollback script. Rollbacks are for the narrow window between apply and first production use. Once a migration has been in production for more than 24 hours, rollback is strongly discouraged — use a forward-fix migration instead.

Rollback 119 (opportunities)

```
-- rollback_119.sql
DROP VIEW IF EXISTS v_renewal_queue CASCADE;
ALTER TABLE sales DROP COLUMN IF EXISTS opportunity_id;
DROP TABLE IF EXISTS interactions CASCADE;
DROP TABLE IF EXISTS opportunities CASCADE;
DROP FUNCTION IF EXISTS stage_weight(opportunity_stage);
DROP TYPE IF EXISTS opportunity_stage;
DROP TYPE IF EXISTS opportunity_channel;
DROP TYPE IF EXISTS opportunity_outcome;
DROP TYPE IF EXISTS interaction_type;
DROP TYPE IF EXISTS interaction_direction;
DROP TYPE IF EXISTS interaction_disposition;
DROP TYPE IF EXISTS interaction_source;
DROP TYPE IF EXISTS interaction_sentiment;
```

Rollback 120 (press trigger)

```
DROP TRIGGER IF EXISTS tr_press_fires_invoices ON issues;
DROP FUNCTION IF EXISTS tf_press_fires_invoices;
DROP FUNCTION IF EXISTS fire_invoice_cascade_on_press;
DROP TABLE IF EXISTS invoice_reminders;
DROP TABLE IF EXISTS integration_jobs;
DROP TYPE IF EXISTS integration_job_type;
DROP TYPE IF EXISTS job_status;
DROP TYPE IF EXISTS reminder_cadence;
DROP TYPE IF EXISTS reminder_status;
ALTER TABLE invoices DROP COLUMN IF EXISTS suppressed_for_credit_hold;
ALTER TABLE invoices DROP COLUMN IF EXISTS partial_press_sent;
```

121, 122, 123, 124 rollback scripts follow the same DROP pattern. Full rollback SQL is generated from migration source via the standard "reverse each DDL" pattern; not included here for brevity.

9.6 Data backfill considerations

Only one migration has a data-backfill concern:

- **Migration 121 — renewal status.** After applying, running run_nightly_renewal_job() once manually will flip eligible Active clients to Renewal status and create opportunities for any client with contract_end_date in the next 60 days. This is the correct one-time backfill. Subsequent runs are incremental.

Expected impact: based on the current dataset of 4,030 clients, approximately 15-40 Active clients should flip to Renewal on first run (estimate from contract_end_date distribution). Fewer than 100 new opportunities will be created. Safe to run in production during normal hours.

9.7 Operational notes

Supabase MCP session limits

- Sessions time out after ~30-40 sequential execute_sql or apply_migration calls. Apply migrations in batches of 2-3 per session, with explicit waits between.
- Long-running migrations (views in 123) may contribute more to the timeout than small ones. Front-load the simpler migrations in each session.
- If a timeout occurs mid-migration, verify state via a fresh SELECT before retrying — partial application is possible.

Compute tier

- Current: Small (recently upgraded from Micro). Sufficient for these migrations.
- Watch for connection pool exhaustion during the first run of run_nightly_renewal_job on production data — it may create 100+ opportunities + interactions in a loop. If observed, wrap the loop in a COMMIT every 50 iterations.

RLS checks

- Every new table (opportunities, interactions, invoice_reminders, integration_jobs, commission_payout_snapshots, integration_credentials) ships with RLS enabled in the migration. Verify policies are present before deploying UI that queries these tables.
- Policies use the pattern: reps see their own rows + publisher/admin sees all. Defined inline in each migration.

9.8 Git commit sequence

Recommended commit structure for the spec-055 branch:

```
spec-055-sales-cycle/
  commit 1: supabase/migrations/119_opportunity_interaction_model.sql
  commit 2: supabase/migrations/120_press_to_invoice_trigger.sql
  commit 3: supabase/migrations/121_renewal_automation.sql
```

```
  commit 4: supabase/migrations/122_commission_payout_execution.sql
  commit 5: supabase/migrations/123_sales_rep_dashboard.sql
  commit 6: supabase/migrations/124_integration_backbone.sql
  commit 7: docs/spec-055-sales-cycle.pdf (this document)
  commit 8-N: Edge Function source + tests
  commit N+1: UI components + routing
  commit final: docs/spec-055-sales-cycle-sign-off.md (acceptance checklist
completed)
```

10. Build Sequence + Claude Code Handoff

Six phases, in order. Each phase is a deployable unit with its own acceptance gate. The build must not skip ahead — later phases assume earlier phases are live and verified. Total realistic timeline: 4-6 weeks of focused Claude Code work, depending on review cadence.

How to use this section

Each phase below lists its migration, Edge Functions, UI components, and definition of done. Paste the phase-specific prompt (Section 10.8) into Claude Code to kick off the build for that phase.

Do not batch phases. Land phase 1 in production (or at least in a staging build), verify, then start phase 2. The dependency chain punishes shortcuts.

When a phase is "done," run the acceptance criteria from that gap's section and the end-to-end smoke test from Section 10.7.

10.1 Phase 1 — Opportunity & Interaction backbone

Builds the core data model from Gap 1. No user-facing UI yet; this phase is pure data infrastructure.

Migration	119_opportunity_interaction_model
Edge Functions	None this phase
UI components	None this phase
RPCs deployed	create_opportunity, advance_opportunity_stage, log_interaction, ingest_gmail_interaction (stub), stage_weight()
Triggers	tr_opportunity_last_interaction, tr_opportunity_weighted_value, tr_proposal_signed_wins_opportunity
Estimated time	1-2 days

Definition of done:

- Migration 119 applied to production Supabase; smoke queries from Section 9.4 pass.
- Manually calling create_opportunity from the SQL editor successfully inserts rows in opportunities + interactions.
- Manually calling advance_opportunity_stage transitions stages, stamps actual_close_date on Won/Lost.
- Inserting into proposal_signatures with signed = true flips the linked opportunity to Won (via trigger).

10.2 Phase 2 — Press cascade + billing foundation

Gap 2. Applies the trigger, deploys the QBO + Stripe workers. Verified end-to-end with a test issue on a dev schedule before going near production press days.

Migration	120_press_to_invoice_trigger
Edge Functions	qbo-invoice-sync, stripe-auto-charge, stripe-webhook, reminder-dispatcher
UI components	Press day confirmation modal (extends existing press button); Integration Jobs dashboard (admin)
RPCs deployed	fire_invoice_cascade_on_press, claim_integration_jobs (from 124 — optional early ship)
Estimated time	1 week

Definition of done:

- Test issue with 3 test invoices: marking sent_to_press_at flips all 3 to sent within 2 seconds.
- One of the test invoices syncs to QBO sandbox with a valid quickbooks_id stored.
- One test client with a Stripe test card gets auto-charged; payment appears in Stripe; webhook creates payments row; invoice flips to paid.
- One test invoice with card_declined test card fails correctly; auto_charge_attempts increments; retry schedule populated in next_attempt_at.
- invoice_reminders rows exist for each sent invoice at 3/14/28 days.

10.3 Phase 3 — Renewal automation

Gap 3. Applies schema changes, deploys the nightly cron, and surfaces the Renewal queue as a standalone page (not yet integrated into the rep dashboard — that's Phase 5).

Migration	121_renewal_automation
Edge Functions	nightly-renewal-runner (cron: 0 10 * * * UTC = 2am Pacific)
UI components	Renewals page (list view of v_renewal_queue, buckets, inline draft button)
RPCs deployed	Updated recompute_client_status, draft_renewal_proposal, run_nightly_renewal_job
Estimated time	3-5 days

Definition of done:

- First manual run of run_nightly_renewal_job() flips all eligible Active clients to Renewal; opportunities created with source_channel = renewal.
- Running it a second time creates no duplicates (idempotent).

- Renewals page shows correct bucket counts matching the view; clicking Draft creates a linked proposal.
- Cron registered and verified to fire at 2am Pacific (log the invocation, inspect activity_log for the completion entry).

10.4 Phase 4 — Commission payout execution

Gap 4. Low code volume, high stakes: this is money moving. The build is the snapshot table + the four RPCs + the publisher Run Composer + the rep Commission tab.

Migration	122_commission_payout_execution
Edge Functions	None (all logic in RPCs)
UI components	Commission Run Composer (3-step modal, publisher); Rep Commission tab (self-service); Past Payouts table; Goal attainment progress bars
RPCs deployed	compose_commission_payout, approve_commission_payout, mark_commission_payout_paid, void_commission_payout
Estimated time	1 week

Definition of done:

- Hayley runs the first real monthly payout for one rep (Dana is a good candidate): composes → reviews → approves → marks paid.
- Snapshot row exists with valid SHA.
- Ledger entries are now status = paid; cannot be included in a subsequent payout.
- Dana logs in, opens her Commission tab, and sees the payout in her history with correct amount, bonus tier, and per-sale breakdown.
- Void test: compose + approve a dummy payout, then void it; ledger entries return to earned status.

10.5 Phase 5 — Sales Rep Dashboard

Gap 5. The capstone UI build. Every prior phase's data now aggregates into one screen.

Migration	123_sales_rep_dashboard
Edge Functions	None
UI components	Full /dashboard page: Revenue Command Bar, Renewal Queue (replaces standalone page), Pipeline Kanban, Today's Actions, Unmatched Comms banner, Recent Activity feed. Publisher variants with rep picker + All Reps aggregation.
RPCs deployed	get_rep_dashboard
Views deployed	v_rep_revenue_kpis, v_rep_today_actions, v_rep_pipeline_kanban,

	v_rep_recent_activity
Estimated time	1.5-2 weeks

Definition of done:

- Dana logs in, lands on dashboard, sees all six zones render in under 800ms.
- KPI numbers reconcile to the underlying tables exactly.
- Drag-drop on Pipeline Kanban updates stage via advance_opportunity_stage with optimistic UI.
- Today's Actions includes at least one item from each source (follow-up, stale, proof, billing, unmatched comm).
- Publisher view with "All Reps" shows correct aggregated numbers.
- Mobile render: zones stack correctly, tab bar works, no horizontal scroll.

10.6 Phase 6 — Integration finalization

Gap 6. The workers exist from Phase 2 (QBO, Stripe); this phase adds Gmail ingest, Calendar sync, credential management UI, and dead-letter queue admin.

Migration	124_integration_backbone
Edge Functions	gmail-ingest, gmail-send, calendar-sync, refresh-oauth-tokens (cron)
UI components	Rep Settings > Integrations page; Publisher Settings > Org Integrations; Dead-letter queue admin; Integration health widget
RPCs deployed	handle_job_failure
Cron jobs	Gmail watch() refresh (every 6 days); token refresh (hourly); Calendar event polling (optional, daily)
Estimated time	1-1.5 weeks

Definition of done:

- Dana connects her Gmail; inbound email from a known client contact creates an interaction row on the correct opportunity within 10 minutes.
- Dana sets a follow-up date on an opportunity; event appears on her Google Calendar with correct title/description.
- Disconnect flow revokes tokens and stops ingest.
- Dead-letter queue admin shows any failed jobs; retry/dismiss buttons work.
- Integration health widget correctly shows green for working integrations, red for any with >10% dead jobs.

10.7 End-to-end smoke test

After all six phases, run this scenario in production with a real-but-small client account:

- **Step 1.** Dana creates a new client (test advertiser) and logs a phone call.
- **Step 2.** Dana creates an opportunity. Confirms it appears on her dashboard Pipeline Kanban.
- **Step 3.** Test advertiser sends Dana an email from their real address. Confirms it appears as an interaction within 10 minutes.
- **Step 4.** Dana builds a proposal from the opportunity. Sends to advertiser.
- **Step 5.** Advertiser clicks the signature link and signs. Confirms: opportunity → Won, contract created, ad_projects row, draft invoices per issue, thread row.
- **Step 6.** Jen designs the ad, sends proof, gets approval. Signoff trigger creates print_placement.
- **Step 7.** Publisher marks the issue sent to press on production. Invoice flips to sent. QBO sync happens. Stripe charges the card on file.
- **Step 8.** Commission ledger entry moves to earned. Dana sees it on her Commission tab.
- **Step 9.** End of month. Hayley runs the payout composer, approves, marks paid. Dana sees it on her Commission history.
- **Step 10.** 60 days before contract end, the client flips to Renewal status, a renewal opportunity is auto-created, and 30 days out an AI-drafted proposal appears. Dana reviews, sends. Loop starts again.

10.8 Claude Code phase-specific prompts

Copy and paste these into Claude Code in the MyDash workspace to kick off each phase. Each assumes the spec document is available in the repo at docs/spec-055-sales-cycle.pdf.

Phase 1 prompt

```
Phase 1 of spec 055 — Opportunity + Interaction backbone.

Reference: docs/spec-055-sales-cycle.pdf, Section 3 and Section 9.1 for migration
119.

Build the following:
1. supabase/migrations/119_opportunity_interaction_model.sql — full SQL from spec
Section 3.7, with all enums, tables, indexes, FK, helper function, and RLS enable +
policies.
2. RPCs from Section 3.4: create_opportunity, advance_opportunity_stage,
log_interaction, ingest_gmail_interaction (stub — returns null match; real logic
ships Phase 6).
3. Triggers from Section 3.5.
4. Apply the migration via supabase db push; run smoke queries from Section 9.4.
```

```
Do NOT build any UI this phase. Commit as one PR with title "spec 055 phase 1:
opportunities + interactions".
Stop and ping for review before moving to Phase 2.
```

Phase 2 prompt

```
Phase 2 of spec 055 — Press cascade + billing.

Reference: docs/spec-055-sales-cycle.pdf, Sections 4 and 8.2-8.3.

Build:
1. supabase/migrations/120_press_to_invoice_trigger.sql (Section 4.5).
2. Edge Functions: qbo-invoice-sync, stripe-auto-charge, stripe-webhook, reminder-
dispatcher (Section 8).
3. Supabase secrets: STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, QBO_CLIENT_ID,
QBO_CLIENT_SECRET.
4. Cron: register each worker at every-minute cadence.
5. UI: press day confirmation modal (extend existing press button in the issue
schedule).
6. UI: Integration Jobs admin table at /admin/integrations/jobs (read-only this
phase).

Verify with a test issue carrying 3 test invoices. Do NOT deploy to production press
days until smoke test passes in staging.
Stop after the smoke test passes. Ping for review.
```

Phase 3 prompt

```
Phase 3 of spec 055 — Renewal automation.

Reference: docs/spec-055-sales-cycle.pdf, Section 5.

Build:
1. supabase/migrations/121_renewal_automation.sql (Section 5.7).
2. Edge Function: nightly-renewal-runner. Cron: 0 10 * * *
3. UI: standalone Renewals page at /renewals (not yet dashboard-integrated).
4. After apply, manually run run_nightly_renewal_job() to backfill statuses +
opportunities.

Stop after first manual run returns expected counts. Ping for review.
```

Phase 4 prompt

```
Phase 4 of spec 055 — Commission payout execution.

Reference: docs/spec-055-sales-cycle.pdf, Section 6.

Build:
1. supabase/migrations/122_commission_payout_execution.sql (Section 6.8).
2. UI: Commission Run Composer (3-step modal) at /admin/commissions/run, publisher-
only.
```

```
3. UI: Rep Commission tab (added to existing rep profile view).
4. UI: Past Payouts table with PDF statement download (generate from snapshot).
5. UI: Goal attainment progress bars on rep dashboard (preview before Phase 5
dashboard exists).

Stop after Hayley runs the first real payout successfully and Dana sees it on her
Commission tab. Ping for review.
```

Phase 5 prompt

```
Phase 5 of spec 055 — Sales Rep Dashboard.

Reference: docs/spec-055-sales-cycle.pdf, Section 7.

Build:
1. supabase/migrations/123_sales_rep_dashboard.sql (Section 7.11).
2. UI: /dashboard page with all six zones (Section 7.1).
3. Integrate existing Renewals page from Phase 3 into the dashboard (remove
standalone route).
4. Publisher variant: rep picker + All Reps aggregation.
5. Performance target: full page render under 800ms with real rep data.

Stop after meeting performance target + all zones populating correctly. Ping for
review.
```

Phase 6 prompt

```
Phase 6 of spec 055 — Integration finalization.

Reference: docs/spec-055-sales-cycle.pdf, Section 8.

Build:
1. supabase/migrations/124_integration_backbone.sql (Section 8.8).
2. Edge Functions: gmail-ingest, gmail-send, calendar-sync, refresh-oauth-tokens.
3. Google Cloud: Pub/Sub topic, Gmail Push subscription setup.
4. UI: Rep Settings > Integrations page (connect/disconnect Gmail, Calendar, toggle
ingest).
5. UI: Publisher Settings > Org Integrations (QBO realm, Stripe key).
6. UI: Dead-letter queue admin with retry/dismiss.
7. UI: Integration health widget on admin dashboard.
8. Replace the Phase 1 ingest_gmail_interaction stub with the real matching
algorithm (Section 3.3.1).

Stop after end-to-end smoke test (Section 10.7) passes with a real test advertiser.
Ping for final review + spec sign-off.
```

10.9 Post-build operations

Once all six phases are live, the following operational cadence takes effect:

- **Daily.** Nightly renewal job runs at 2am Pacific. Integration workers run continuously. Dashboard refreshed live.
- **Weekly.** Gmail watch() refresh cron. Dead-letter queue review.
- **Monthly.** Commission payout run (or bi-weekly per rep's team_members.commission_payout_frequency setting). Integration health review.
- **Quarterly.** Rate card + renewal pricing rule review (org_settings.renewal_pricing_rules). Bonus tier thresholds review.

10.10 Future work (out of scope for this spec)

Captured here to make the boundary explicit. These are the logical next specs after 055 lands:

- Subscription management module (replaces SimpleCirc).
- Circulation/distribution module (racks, hotels, businesses).
- Wednesday Agent Station integration (Gemma 4 26B agents).
- Self-hosted CDN feasibility (replacing BunnyCDN).
- Publisher Command Center dashboard (all-reps, all-departments view).
- Role-based dashboards for the four non-sales roles (Content Editor, Layout Designer, Ad Designer, Office Admin).
- QuickBooks payroll sync for commission payouts (currently manual).
- AI-powered email draft assistance (opportunity context → outbound email).
- Historical sales backfill into the opportunity model (if ever needed for reporting parity).

End of spec

This document is the complete specification for MyDash spec 055 — Sales Cycle. It is designed to be handed to Claude Code in the IDE as the single source of truth for the build.

Authored by Claude (desktop) in collaboration with Andrew Jackson, Editor-in-Chief, 13 Stars Media Group, April 2026.

Next step: begin Phase 1.